



Szegedi Tudományegyetem
Természettudományi és Informatikai Kar
Informatikai Intézet
Szoftverfejlesztés Tanszék

Szakdolgozat

Önvezető jármű vezérlésének megvalósítása virtuális
kétdimenziós környezetben, mesterséges intelligencia
alkalmazásával

Készítette:
Simon Bence
Programtervező informatikus
BSc hallgató

Témavezető:
Dr. Dombi József Dániel
Szoftverfejlesztés Tanszék
adjunktus

Szeged
2024

Feladatkiírás

A szakdolgozatom célja egy olyan mesterséges intelligencia (MI) modell kifejlesztése, amely képes egy járművet irányítani virtuális környezetben. A modell a “sofőr” szemszögéből kapott képi információkat használja fel, hogy meghatározza a megfelelő irányítási parancsokat.

A dolgozathoz a Python programozási nyelvet fogom használni, ami nagyon fontos szerepet játszik a mesterséges intelligencia fejlesztésben. Könnyen tanulható, hatékony, magasszintű adatstruktúrát, objektum-orientáltságot, dinamikus típusosságot kínál a fejlesztőknek, kutatóknak a természettudományok területén is. Nem beszélve a megszámlálhatatlan modullról, ami a fejlesztők rendelkezésére áll.

A pálya megvalósításához a PyGame nevű Python modult fogom használni, amely alkalmas egy kétdimenziós tér elkészítéséhez. A felülnézetes pálya útból, falakból és különböző akadályokból állhat, például sár és egyéb nehezítések. A felhasználó képes befolyásolni az autó sebességét, kanyarodási képességét, valamint a pálya bizonyos részeinek törlését.

Az “önvezetés” egy mesterséges neurális hálóval lesz megvalósítva. A mesterséges neurális hálózat olyan számítógépes rendszereket takar, amely a biológiai idegrendszert hivatottak utánózni. Az emberi agy neuronoknak nevezett idegsejtekből áll, a mesterséges neurális hálók is hasonlóan működnek. Céljuk a gépi tanulás, ami a tanuló rendszerek gyakorlati alkalmazása.

A neurális hálózatok felhasználási területe nagyon széleskörű, ezek közül a kiemelném szakdolgozatom témaként is választott, a járműveket is vezérelni képes, egyébként rengeteg más területen használt, backpropagation (hiba-visszaterjesztéses) algoritmusokat.

A backpropagation algoritmus egy olyan módszer, amely a gradienst csökkenést (gradient descent) használja a mesterséges neurális hálózatok felügyelt tanulásához. Adott egy mesterséges neurális hálózat és egy hibafüggvény, az eljárás kiszámítja a hibafüggvény gradienst a neurális hálózat súlyaihoz képest. A folyamat során a hibát visszavezetjük a hálózat rétegein keresztül, hogy finomhangoljuk a súlyokat. Ez azt jelenti, hogy a hálózat súlyait az előző iteráció hibája alapján állítjuk be. A súlyok megfelelő beállítása csökkenti a hibát és növeli a modell megbízhatóságát. Egyszerűen fogalmazva, a backpropagation segít a neurális hálózatoknak “tanulni” és javítani teljesítményüket.

A szakdolgozat bemutatja a modell architektúráját, a tanulási módszertant, az alkalmazott technológiákat, valamint az eredményeket és az értékelést. További célja, hogy bemutassa, hogy az MI modell hogyan képes adaptálni és javítani a vezetési teljesítményét különböző körülmények között.

Tartalmi összefoglaló

A téma megnevezése:

Önvezető jármű vezérlésének megvalósítása virtuális kétdimenziós környezetben, mesterséges intelligencia alkalmazásával

A feladat megfogalmazása:

A feladat egy olyan Python alkalmazás elkészítése, amely mesterséges intelligencia segítségével vezérel egy autót. A pálya szerkeszthető legyen különböző akadályokkal, és több úttípust is tartalmaz. Az autó szemszögét is látni lehet, és az autó irányítását is lehet befolyásolni különböző paraméterekkel. Az autó célja a pályán maradás. Tartalmazza a tanítási folyamatot is.

Megoldási mód:

A feladatot Python nyelven oldottam meg. A PyGame könyvtár segítségével valósítottam meg a pálya elemeinek betöltését, maszkolását, vezérlőpanelt és eseménykezelést. Az autó szemszögét szenzorok biztosítják, és az autó paraméterei beállíthatóak. A gépi tanulási módszerek közül egy neurális hálózatos és egy Q-táblás megoldást választottam, amit végül összehasonlítottam.

Alkalmazott eszközök, módszerek:

PyCharm fejlesztői környezettel Python 3.9-cel oldottam meg a feladatot. A pályát a PyGame modullal, az MI részt TensorFlow, Keras, és scikit-learn modulokkal, az adatfeldolgozást a Pandas, NumPy modulokkal oldottam meg. Az autó irányításához használtam még a Math modult. A tesztelés kiértékelését és vizualizációjához a Matplotlibet.

Elért eredmények:

Sikeresen létrehoztam egy 2D pályát, ahol különböző akadályok és úttípusok találhatók. Az autó paraméterei befolyásolhatóak. Az autó képes a pályán maradni, akadályokat kikerülni két különböző gépi tanulás technikán.

Kulcsszavak:

python, mesterséges intelligencia, gépi tanulás, önvezetés, neurális hálózat, q-learning

Tartalomjegyzék

1. Bevezetés	5
2. Python szerepe az MI fejlesztésben	7
2.1. Python jellemzői és előnyei az MI szempontjából	7
2.2. Python modulok és eszközök az MI fejlesztésben	8
2.2.1. TensorFlow	8
2.2.2. Keras	8
2.2.3. Numpy	9
2.2.4. Pandas	10
3. Pálya megalkotása	11
3.1. PyGame	11
3.2. PyGame beállítása	11
3.2.1. Pálya elemeinek betöltése	11
3.2.2. Maszkolás	12
3.2.3. Vezérlőpanel	13
3.2.4. Eseménykezelés és a Fő ciklus.....	14
3.2.5. Obstacle osztály.....	15
4. Jármű megjelenítése, vezérlői megoldások	17
4.1. Car osztály	17
4.1.1. A jármű tulajdonságai	17
4.1.2. A jármű mozgatása	19
4.2. Szenzorok bevezetése	22
5. Gépi tanulási módszerek	25
5.1. Gépi tanulás	25
5.1.1 Gépi tanulás alkalmazásai	26
5.1.2 Gépi tanulás kategóriái	27
5.1.3 Overfitting és Underfitting	28
5.2. Neurális hálózatok	30
5.2.1. Bias.....	31
5.2.2. Aktivációs függvény.....	32
5.2.3. Veszteségfüggvények	33
5.2.4. Feed-forward féle neurális hálózat	34
5.3. Q-tábla	36

5.3.1. A Q-tábla alapjai	37
5.3.2. A Markov döntési folyamatok	37
5.3.3. A Q-tábla frissítési szabálya	38
6. SL és RL módszerek elkészítése	40
6.1. SL modell: FNN	40
6.1.1. Adatgyűjtés	40
6.1.2. Az adatok előkészítése	41
6.1.3. Az FNN felépítése	42
6.2. RL módszer: Q-learning	43
6.3. Irányítások összehasonlítása	45
7. Tanítási folyamat és kiértékelés	47
7.1. SL modell tanítása	47
7.1.1. SL modell eredményei	47
7.2. A Q-learning tanítása és eredményei	50
7.3. Összegzés	51

1. Bevezetés

A szakdolgozatom célja egy olyan szimulált mesterséges intelligencia (MI) modell által vezérelt jármű kifejlesztése, amely képes irányítani azt egy virtuális környezetben. A modell a “sofőr” szemszögéből szenzorok alapján kapott információkat használja fel, hogy meghatározza a megfelelő irányítási parancsokat.

A dolgozathoz a Python programozási nyelvet fogom használni, ami nagyon fontos szerepet játszik a mesterséges intelligencia fejlesztésben. Könnyen tanulható, hatékony, magasszintű adatstruktúrát, objektum-orientáltságot, dinamikus típusosságot kínál a fejlesztőknek, kutatóknak a természettudományok területén is. Nem beszélve a megszámlálhatatlan modulról, ami a fejlesztők rendelkezésére áll.

A pálya megvalósításához a PyGame nevű Python modult fogom használni, amely alkalmas egy kétdimenziós tér elkészítéséhez. A felülnézetes pálya útból, pálya szélből és különböző akadályokból állhat, például sár és egyéb nehezítések. A felhasználó képes befolyásolni az autó sebességét, kanyarodási képességét, valamint a gyorsulást és a fékezést.

Az önvezetés egy mesterséges neurális hálóval lesz megvalósítva. A mesterséges neurális hálózat olyan számítógépes rendszereket takar, amely a biológiai idegrendszert hivatottak utánozni. Az emberi agy neuronoknak nevezett idegsejtekből áll, a mesterséges neurális hálók is hasonlóan működnek. Céljuk a gépi tanulás, ami a tanuló rendszerek gyakorlati alkalmazása.

A mesterséges neurális hálózatok az emberi agy működését szimuláló számítási modellek, amelyek a mesterséges intelligencia és gépi tanulás területén jelentős áttörést hoztak. Strukturálisan rétegekből állnak, ahol minden rétegben több neurális egység található, amelyek a biológiai neuronokhoz hasonlóan továbbítják az információt a következő rétegekbe.

A neurális hálózatok képesek összetett mintázatok és összefüggések felismerésére nagy mennyiségű adatban, legyen az képfeldolgozás, beszédfelismerés vagy természetes nyelvfeldolgozás. A hálózatok megtanulják ezeket a mintákat egy olyan folyamaton keresztül, amelyet tanításnak nevezünk, és amely során egy hibafüggvény segítségével optimalizálják a belső paramétereiket, az úgynevezett súlyokat. Ez a képességük teszi a mesterséges neurális hálózatokat nélkülözhetlenné olyan feladatoknál, amelyek az emberi intuíciót és megértést igénylik.

A szakdolgozat bemutatja a modell architektúráját, a tanulási módszertant, az alkalmazott technológiákat, valamint az eredményeket és az értékelést. További célja, hogy

bemutassa, hogy az MI modell hogyan képes adaptálni és javítani a vezetési teljesítményét különböző körülmények között.

2. Python szerepe az MI fejlesztésben

Python a mesterséges intelligencia fejlesztéseknek az egyik legnépszerűbb, általános célú programozási nyelve, amely számos előnyt kínál, amikor komplex adatokkal dolgozunk vagy gépi tanulási modelleket építünk. A nyelv egyszerű szintaxisa, kiterjedt könyvtár ökoszisztémája és széles körű támogatása olyan előnyöket kínál, amelyek jelentősen hozzájárulnak az MI fejlesztések sikereihez. Ebben a fejezetben megnézzük részletesebben ezeket az előnyöket.

2.1. Python jellemzői és előnyei az MI szempontjából

A szintaxis kifejezetten intuitív, könnyen tanulható és lehetővé teszi a fejlesztők számára, hogy a problémamegoldásra, az algoritmusok tervezésére és optimalizálására koncentráljanak a kódolás helyett. Ez különösen fontos az MI-fejlesztésben, ahol a gyors prototípus-készítés és a komplex algoritmusok implementálása gyakori feladat. Az egyszerű szintaxis lehetővé teszi, hogy a kutatók és fejlesztők könnyebben megértsék egymás kódját, ami növeli a hatékonyságot és csökkenti a kommunikációs akadályokat.

Interpretált nyelv, tehát a kód futtatása során a forráskódot közvetlenül végrehajtható utasításokká alakítja. Ez a tulajdonsága lehetővé teszi a gyors prototípuskészítést és a hibakeresést, mivel a kód egyes részei külön-külön is tesztelhetők anélkül, hogy minden változtatásnál újra kellene fordítani az egész programot.

A dinamikus típuskezelése rugalmas fejlesztést tesz lehetővé. A gépi tanulási modellek prototípusának fejlesztése során gyakran változtatják a modell struktúráját és paramétereit. A Python dinamikus típuskezelése egyszerűsíti az ilyen módosításokat, lehetővé téve, hogy a fejlesztők gyorsan reagáljanak az új követelményekre vagy kutatási eredményekre, így felgyorsítva az iteratív fejlesztési folyamatot.

A Python magas szintű absztrakciókat támogat, amelyek megkönnyítik a fejlett gépi tanulási algoritmusok implementálását. Ez a támogatás még hatékonyabbá válik a széles körben használt gépi tanulási keretrendszerek, például a TensorFlow és a PyTorch használatával, amelyek kiterjedt funkcionalitást kínálnak Python API-kkal. Ezekkel az absztrakciós rétegekkel a fejlesztők egyszerűen létrehozhatnak mélytanulási modelleket anélkül, hogy a hardveres vagy alacsony szintű implementációs részletekkel foglalkoznának.

A Python híres a széleskörű modul támogatásáról, amely rendkívül gazdag ökoszisztémával rendelkezik. Számos előre elkészített modult kínál különböző adatkezelési és

gépi tanulási feladatokhoz. Ilyen például a **NumPy**, amely a numerikus adatok kezelésére használatos, a **Pandas** az adatok manipulálására és elemzésére, a **Matplotlib** adatvizualizációra, a **TensorFlow** és a **Keras**, pedig két kulcsfontosságú keretrendszer a mélytanulás területén.

A Python egyik legnagyobb előnye a hatalmas globális közösség, mely aktívan részt vesz a nyelv fejlesztésében, támogatásában. A közösség által létrehozott dokumentációk, fórumok segítik a kezdőket és a haladókat egyaránt. A problémák gyors megoldása, a tapasztalatok megosztása, és az új eszközök folyamatos bevezetése biztosítja, hogy a Python programozók a legfrissebb technológiákat és módszereket használhassák.

A Python platformfüggetlensége azt is biztosítja, hogy az alkalmazások különböző operációs rendszereken fussanak, beleértve a Windows, macOS és Linux rendszereket. A cloud szolgáltatások növekedésével a fejlesztők rugalmasan választhatják meg a fejlesztési és telepítési környezetet.

2.2. Python modulok és eszközök az MI fejlesztésben

Az MI fejlesztésekor számos nagyteljesítményű modul áll rendelkezésre. Az általam használt kódban több olyan könyvtárt használok, amik az MI fejlesztést támogatják. Ilyen például a **TensorFlow**, a **Pandas**, a **Keras**, és a **NumPy** is, amelyekről most részletesebben fogok írni.

2.2.1. TensorFlow

A TensorFlow egy nyílt forráskódú gépi tanulási keretrendszer, amelyet a Google fejlesztett ki [2]. Az egyik legelterjedtebb platform a mélytanulási alkalmazásokhoz, mivel rugalmas és méretezhető architektúrával rendelkezik, így nagy mennyiségű adaton is hatékonyan működik. A TensorFlow elsősorban a tensorok kezelésén alapul, ahol a tensorok többdimenziós tömbök vagy mátrixok formájában reprezentálják az adatokat. Ezeken a tensorokon matematikai műveleteket végeznek a hálózati rétegek, ami az egész rendszer nevét is inspirálta. Az architektúrája lehetővé teszi a GPU-k és TPU-k teljes kapacitásának kihasználását, így az alkalmazások skálázhatók és nagy számítási igényű projektekben is használhatóak. Ezenkívül támogatja a különböző modellkészítési stratégiákat, például a szekvenciális és funkcionális API-kat, amelyek rugalmas modellezési lehetőségeket kínálnak.

2.2.2. Keras

A Keras egy magas szintű mélytanulási keretrendszer [3], amely ma már a TensorFlow része. Eredetileg független projektként indult, célja pedig az volt, hogy felhasználóbarát, egyszerű és gyors eszközöket kínáljon a neurális hálózatok tervezéséhez és fejlesztéséhez. A Keras moduláris felépítése lehetővé teszi a fejlesztők számára, hogy rétegenként építsenek mélytanulási modelleket, egyszerűen kombinálva különböző rétegtípusokat, a kívánt hálózat létrehozásához. Például: Dense, Dropout, Conv2D. Az API-k egyszerűek és intuitívak, így a fejlesztők a modellezési és optimalizálási feladatokra koncentrálhatnak ahelyett, hogy a belső működés részleteivel kellene foglalkozniuk. A Keras továbbá olyan fejlett képességeket kínál, mint a több kimeneti modell támogatása, az előre beállított aktivációs függvények és optimalizálók, valamint a korai leállítást vagy a tanulási ráta csökkentése.

Az optimalizáló algoritmusok kulcsfontosságú szerepet játszanak a gépi tanulási modellek betanításában, mivel ezek határozzák meg, hogyan frissüljenek a modell súlyai az egyes tanítási ciklusok során. Az optimális súlyok megtalálása érdekében az algoritmusok folyamatosan finomítják a modell paramétereit a veszteségfüggvény minimalizálása érdekében. A Keras többféle optimalizálót is kínál, amelyek közül a három leggyakrabban használt az SGD, az RMSprop és az Adam.

Az SGD a klasszikus gradiens-alapú optimalizáló algoritmus egyik változata. A módszer lényege, hogy a teljes adathalmaz helyett csak kis adatcsoportok úgynevezett mini-batch-ek alapján számítja ki a gradiensértékeket, így gyorsabbá és hatékonyabbá válik az optimalizáció, különösen nagy adathalmazok esetében. Az SGD emellett támogatja a momentum paramétert, amely az előző lépés irányát és sebességét figyelembe véve segít túllépni a lokális minimumpontokon.

Az RMSprop egy adaptív tanulási rátájú algoritmus, amely különböző jellemzők esetében automatikusan változtatja a tanulási sebességet. Az RMSprop a gradiens négyzetes átlagát használja az egyes súlyok tanulási rátájának szabályozására. Így az algoritmus alkalmazkodik a különböző változók eltérő skálájához, ami jelentősen javítja a konvergenciát.

Az Adam kombinálja az RMSprop és az SGD momentum megközelítésének előnyeit. Az algoritmus egyszerre tartja számon a gradiens első és második momentumát is, így a tanulási ráta dinamikusan és adaptívan változik az egyes paraméterek szerint. Az Adam gyors és hatékony konvergenciát kínál, ezért az egyik legnépszerűbb optimalizáló algoritmus mélytanulási alkalmazásokban.

2.2.3. NumPy

A NumPy egy hatékony Python-könyvtár, amely támogatja a többdimenziós tömbök és mátrixok gyors számítását [4]. Az adatszerkezetek mellett széles körű matematikai függvényeket biztosít, így hatékony adatkezelést és elemzést tesz lehetővé. A NumPy tömbjei (`ndarray`) hasonlóak a Python listákhoz, de hatékonyabbak, mivel homogén adattípusokat tartalmaznak, és előre lefoglalt memóriaterülettel működnek. Ennek köszönhetően a NumPy műveletei gyorsabbak, mint a hagyományos Python listákon végrehajtott számítások. Emellett számos beépített függvényt biztosít lineáris algebrai és matematikai számításokhoz, beleértve a mátrixműveleteket, valamint az olyan egyszerűbb műveleteket, mint a négyzetgyök, szinusz és koszinusz függvények. A NumPy tömbök hatékony indexelést és szeletelést is lehetővé tesznek, amely nagy adathalmazok kezeléséhez és manipulálásához elengedhetetlen. Továbbá a könyvtár kiválóan integrálható más, magas teljesítményű könyvtárakkal, mint a SciPy vagy a Pandas, így tökéletesen illeszkedik a tudományos számítások és adatfeldolgozás ökoszisztémájába.

2.2.4. Pandas

A Pandas egy másik, kiemelkedően hatékony Python-könyvtár, amelyet strukturált, táblázatos adatok kezelésére és elemzésére terveztek [5]. Legfőbb adatszerkezetei a `DataFrame` és a `Series`. A `DataFrame` egy kétdimenziós, oszlop-alapú adatszerkezet, amely lehetővé teszi a különböző adatokat tartalmazó oszlopok kombinációját. Az adatok különböző típusúak lehetnek, és a `DataFrame` támogatja az indexelést, szűrést és rendezést. A `Series` pedig egy egydimenziós adatszerkezet, amely alapvetően egy egyedi indexű lista és egy-egy `DataFrame` oszlopot vagy egy egyszerű, egydimenziós adatsort képviselhet. A Pandas kiválóan kezeli a hiányzó adatokat, különböző forrásokból, például CSV, Excel vagy SQL táblákból olvas és ír adatokat. Számos beépített függvénnyel rendelkezik az adatok előfeldolgozásához, tisztításához és átalakításához, valamint támogatja a statisztikai és idősoros elemzéseket.

3. Pálya megalkotása

3.1. PyGame

Ebben a fejezetben látott kódok a [6] alapján készültek. A PyGame egy Python-modul, amelyet kifejezetten játékfejlesztésre terveztek. Lehetővé teszi, hogy a programozók könnyedén készítsenek 2D-s játékokat és szimulációkat, mivel biztosítja az összes szükséges funkciót a grafika, hang és felhasználói interakciók kezeléséhez. Az egyik legfontosabb eleme a PyGame-nek az ablak kezelése, amelynek segítségével a fejlesztők saját grafikai elemeket helyezhetnek el és különféle interakciókat valósíthatnak meg a játékosok számára.

3.2. PyGame beállítása

A PyGame könyvtár teljes funkcionalitásának használatához szükséges a `pygame.init()` függvény meghívása, amely inicializálja a belső modult. Ez biztosítja, hogy a PyGame rendelkezésre álló összes komponense (mint például az ablakkezelő) elérhető legyen.

Az ablak méretének beállításával kezdünk, ennek méreteit a `WIDTH` – szélesség, és a `HEIGHT` – magasság változók adják meg. Az ablak dimenziói jelen esetben 1200x900 pixel méretűek és az ablak nem újraméretezhető. A megjelenítő felület létrehozásához használjuk a `pygame.display.set_mode()` függvényt, amely egy szélesség és magasság paramétert vár és hozza létre az ablakot. Ezt követően beállítjuk az ablak címét a `pygame.display.set_caption()` függvénnyel amely egy stringet vár, ami a program fejlécében jelenik meg.

3.2.1. Pálya elemek betöltése

A különböző grafikai elemek, mint a fű, a pálya, az autó és az akadályok betöltése a beépített `pygame.image.load` függvénnyel valósul meg. Mivel a képeket mindenképpen át kell méreteznünk, ezért egy kiegészítő függvényt definiálunk, ami a `scaling()` nevet kapta és a 3.1-es kódrészlet mutatja be.

Paraméterei a skálázandó kép és a skálázás mértéke. A függvény meghatározza az új méretet az eredeti kép skálázásával és a beépített `round()` függvénnyel kerekítjük, hogy biztosan egész számot kapjunk. A `pygame.transform.scale()` függvénnyel történik meg maga a transzformáció, ami a visszatérési értéke lesz a metódusnak és paraméterként az eredeti képet várja illetve az új méretet.

```

1 def scaling(img, scale):
2     new_size = round(img.get_width() * scale),
      round(img.get_height() * scale)
3     return pygame.transform.scale(img, new_size)

```

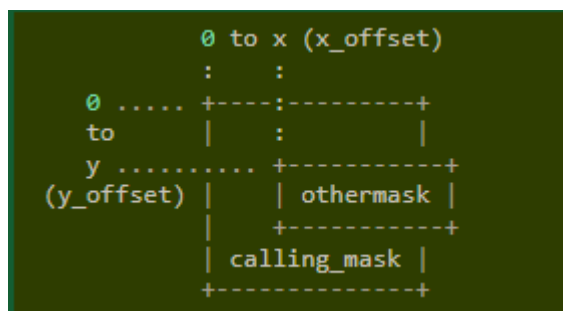
3.1. kódrészlet: scaling metódus a betöltött képek átméretezésére szolgál

3.2.2. Maszkolás

A következő lépés a maszkok elkészítése. A legfontosabb elem, amit az ütközések pontos felismerése érdekében használunk a program során. Maszkokat a `pygame.mask.from_surface()` funkcióval már egy meglévő grafikai elem (Surface) alapján hozzuk létre, amellyel könnyen detektálhatjuk az egyes objektumaink ütközését. Ez a függvény a megadott képen található nem átlátszó pixelek alapján generál egy bitmátrixot, ahol a nem átlátszó területeket 1-ként, az átlátszókat 0-ként tárolja.

Az ütközésvizsgálat egyik legfontosabb maszkolási művelete a `mask.overlap` függvény, amely egy adott pozíció eltolásával két maszk közötti metszetet határozza meg. Ha a maszkok metszete nem üres, akkor a két objektum ütközik egymással.

- Eltolás: A 3.1-es ábrán látható [6] az eltolás, amelyen az egyik maszk bal felső sarka és a másik maszk pozíciójának különbsége határozza meg.



3.1. ábra [6]: eltolás

Tegyük fel, hogy két maszkot hasonlítottunk össze a saját pozícióikhoz képest, 3.2-es kódrészlet:

```

1 car_mask = pygame.mask.from_surface(car_image)
2 offset = (int(car_position[0] - obstacle_position[0]),
      int(car_position[1] - obstacle_position[1]))
3 if car_mask.overlap(obstacle_mask, offset):
4     print("Ütközés történt!")

```

3.2. kódrészlet: Példa kód az ütközés szemléltetéséhez

A maszkolási mátrix pontosan a kép méretéhez igazodik, így a maszk által adott válasz is pixelpontosságú lesz. Ez teszi a maszkolást hatékonyabbá és pontosabbá az egyszerű téglalap alakú ütközésvizsgálathoz képest.

3.2.3. Vezérlőpanel

A 3.2-es ábra, vagyis a vezérlőpanel főleg a későbbiekben lesz fontos, ugyanis itt lesz megtalálható az autó tulajdonságait szabályzó csúszkák is. A vezérlőpanel egy különálló grafikus felületként, úgynevezett Surface-ként jelenik meg a játékalak jobb oldalán. Ehhez létre kell hozni egy felületet a `pygame.Surface` segítségével, amely megfelelő méretű ahhoz, hogy minden fontos adat és interaktív elem elférjen rajta.

A panelen található interaktív elemeket és szövegeket a `blit` metódussal helyezzük el, amelyek információkat jelenítenek meg az autó aktuális állapotáról vagy lehetővé teszik a paraméterek módosítását. A panelen olyan statikus információk jelennek meg, mint a jelenlegi sebesség, maximális sebesség és gyorsulás. Ezeket a `pygame.font` modullal készített szövegek formájában jelenítjük meg.

A csúszkákkal a felhasználó beállíthatja az autó maximális sebességét, forgási sebességét és gyorsulását. Ezek az elemek a `pygame_widgets` csomagban található `Slider` osztály segítségével jönnek létre. A csúszkák paraméterei tartalmazzák a pozíciót, a méretet és az értéktartományt, így a játékos könnyen módosíthatja az autó különböző tulajdonságait. Ne felejtsük el hozzáadni a fő ciklushoz a csúszkák eseményfigyelését se a `slider.listen()` függvénnyel, illetve a frissítést pontosan megjeleníteni a `slider.draw()` funkcióval



3.2. ábra: a vezérlőpanel

A gombokkal a vezérlőpanel lehetőséget biztosít a játékosnak az akadályok hozzáadására vagy eltávolítására is. A gombok, például egy akadálytörő gomb, egyszerű, színes téglalapokként rajzolhatóak meg a `pygame.draw.rect` függvénnyel, és egy szöveget is társítanak hozzájuk a gomb funkciójának jelzésére. [A vezérlőpanel és a gombok kódja a 2sz_melleklet_vezerlopanel.txt fájlban találhatóak.](#)

A panelen megjelenített gyalogos [11] és sár [12] ikonok segítségével új akadályokat tudunk hozzáadni a pályához nehezítésképpen. A `random_position_within_mask()` segítségével ami a 3.3-as kódrészletben látható, véletlenszerű koordinátákat generálunk egy maszk alapján, ami ebben a programban a pálya maszkja lesz és helyezzük azon belülre. Paraméternek a maszk kell, amin belülre lerakjuk az akadályt, majd ennek méretei szükségesek. Ezután lekérjük a maszk méreteit is és egy ciklussal elhelyezzük az akadályt egy olyan pontra ahová még nem került. Visszatérési érték a két szabad koordináta.

```
1 def random_position_within_mask(mask, image_width, image_height):
2     mask_width, mask_height = mask.get_size()
3     while True:
4         x = random.randint(0, mask_width - image_width)
5         y = random.randint(0, mask_height - image_height)
6         if mask.get_at((x, y)) != 0:
7             return x, y
```

3.3. kódrészlet: Kód a véletlenszerű pont generálásához egy maszkon belül

3.2.4. Eseménykezelés és a Fő ciklus

A `while True` elindít egy végtelen ciklust, ami folyamatosan futni fog egészen addig, amíg valamilyen feltétel, ez a 3.4-es kódrészletben található, például a program leállítása nem változtatja meg azt. Az eseménykezelés az interaktív programok egyik kulcsfontosságú komponense, ez biztosítja a felhasználói bemenetek – billentyűk, egérg kattintások – kezelését. A programban a PyGame által biztosított eseménykezelési mechanizmus, egy eseményfigyelő ciklus segítségével valósul meg. A PyGame által generált összes eseményt a `pygame.event.get()` függvénnyel kapjuk meg, amely egy listát ad vissza a jelenleg sorban álló eseményekről. Az események típusa változó, például a billentyűzet, egér vagy rendszereseményeket is tartalmazhat. Az események feldolgozása egy egyszerű cikluson keresztül történik, amely egyenként, sorban iterál végig a begyűjtött eseményeken. Az egyes

események a `pygame.event` modulban definiált állandó értékekkel kerülnek reprezentálásra, így az adott eseménytípus az `event.type` attribútumból nyerhető ki. Például a következő sor a `pygame.QUIT` típusú eseményt azonosítja, amely akkor következik be, amikor a felhasználó a kilépés gombra kattint. Ez a kódfejezet biztosítja, hogy a kilépéskor a program korrekt módon záruljon be, felszabadítva a PyGame által foglalt erőforrásokat.

```
1 while True:
2     events = pygame.event.get()
3     for event in events:
4         if event.type == pygame.QUIT:
5             pygame.quit()
6             sys.exit()
```

3.4. kódrészlet: A szimuláció végtelen ciklusa, és eseménykezelője

A `pygame.KEYDOWN` és `pygame.KEYUP` típusú események kezelik a billentyűk lenyomását és felengedését. Ezek az események használhatók például a jármű irányítására a játékban. Az egérekattintásokat (pl. `pygame.MOUSEBUTTONDOWN`, `pygame.MOUSEBUTTONUP`) és az egér mozgását (`pygame.MOUSEMOTION`) külön eseménytípusok kezelik. A kódban az akadályok vezérlésénél fontos szerepet kapnak. A csúszkák (és egyéb widgetek) állapotának frissítéséhez és kirajzolásához a `pygame_widgets.update` függvény biztosít egy általános frissítési mechanizmust. Ez a függvény gondoskodik arról, hogy az összes csúszka, gomb vagy egyéb interaktív elem reagáljon a felhasználói bemenetekre. Az eseménykezelés a játék folyamatos működésének és a felhasználóval való interakciójának alapja. A PyGame eseményfigyelő ciklusa és az `event.type` változókra épített feltételes logika gondoskodik arról, hogy a program minden releváns felhasználói vagy rendszereseményre megfelelően reagáljon.

3.2.5. Obstacle osztály

Az Obstacle osztály összes metódusa megtekinthető az `'obstacles.py'` fájlban

Az Obstacle osztály egy akadályok létrehozására és kezelésére szolgáló komponens, amely a PyGame segítségével biztosítja ezek grafikus megjelenítését és interaktív manipulálását. Az akadályok mozgathatóak a játéktérületen, és összehangolhatók az autó érzékelőivel, illetve más pályaelemekkel. Az osztály fő funkciói közé tartozik az akadályok grafikus megjelenítése, az ütközések detektálása és az egérrel történő mozgítás kezelése.

A konstruktor az akadály képi reprezentációját (*image*) és kezdeti pozícióját (*position*) kapja paraméterként. A képből létrehozza a PyGame által biztosított téglalap-alakú keretet

(*rect*) a pozícióval (*opleft*). A *dragging* attribútum jelzi, hogy az akadályt éppen mozgatják-e. Az eltolásokat az *offset_x* és *offset_y* attribútumok tárolják. A maszkot a `pygame.mask.from_surface` segítségével hozza létre, hogy az akadály nem átlátszó részeit pontosan meg lehessen határozni a grafikus ütközésvizsgálat során.

A `collides_with_point()` függvény ellenőrzi, hogy egy adott pont benne van-e az akadály maszkjában, azaz az akadályt ábrázoló képen lévő nem átlátszó pixel része-e. A bemeneti pont (*point*) és az akadály pozíciója alapján kiszámítja az eltolást (*offset_x* és *offset_y*), majd ellenőrzi, hogy az adott pontban lévő pixel nem átlátszó-e.

Az `handle_event()` függvény az egérrel kapcsolatos események kezelésére szolgál, hogy az akadályokat a felhasználó a felületen mozgathassa. Ha az akadályra kattintanak (`MOUSEBUTTONDOWN`), a *dragging* attribútum `True` értékre áll, és az eltolásokat a kattintás pozíciója alapján kiszámítja. Ha a felhasználó elengedi az egérgombot (`MOUSEBUTTONUP`), a *dragging* értéke `False` lesz, így megszűnik az akadály húzása. Ha az egér mozog (`MOUSEMOTION`), és az akadályt éppen húzzák, akkor frissíti az akadály pozícióját az egér aktuális helyzetének megfelelően.

4. A jármű megjelenítése, vezérlői megoldások

A kód egyik központi eleme az önvezető jármű, amelyet a `Car` osztályban definiáltak. Ez az osztály felel az autó szimulált környezetben való viselkedésének, mozgásának, és érzékelőinek (szenzorainak) irányításáért. Minden releváns műveletet kezel, beleértve a forgatást, az előre- vagy hátramenetet, a lassítást és az érzékelők használatát, hogy valósághű és játékosbarát élményt nyújtson.

4.1. A Car osztály

4.1.1. A jármű tulajdonságai

A `Car` osztály az autó szimulációjában a jármű legfontosabb tulajdonságait és viselkedését definiálja. Ez az osztály felelős a grafikus megjelenítésért, az interaktív vezérlésért és a környezettel való kölcsönhatásért, beleértve az akadályok elkerülését és a szenzoradatok használatát. Nézzük meg részletesen a jármű felépítését és működését:

Az előző fejezetben bemutatott `pygame.image.load()` függvénnyel betöltjük az autót, majd méretezése után beállítjuk az autó kezdő pozícióját. `START_POSITION` definiálja a jármű induló helyzetét, jelen esetben `x, y` koordináták, hogy mindig azonos pozícióból induljon.

A jármű sebességének és forgási tulajdonságainak részletes vizsgálata kulcsfontosságú a szimulált környezetben történő navigációhoz. Ezek az értékek határozzák meg, hogyan reagál a jármű a felhasználói vezérlésre vagy a gépi vezérlésre, és biztosítják az életszerű és pontos mozgást.

1. `max_vel`: (max velocity) Az autó maximális sebessége
 - Ez a változó meghatározza a jármű végsebességét. A szimuláció során a jármű soha nem fogja túllépni ezt az értéket. Ez megakadályozza, hogy az autó kezelhetetlenné váljon nagy sebességnél, és lehetővé teszi a jármű stabil irányítását különböző terepviszonyok között.
 - Például, ha a jármű terepre hajt, akkor az érték lecsökken, hogy az autó ne gyorsuljon túlzottan és ezzel büntesse a járművet a nem szabályos közlekedés miatt.
2. `rotation_vel`: (rotation velocity) A jármű forgási sebessége
 - Ez a változó határozza meg, hogy az autó milyen gyorsan fordul. Az értéket fok/másodpercben adjuk meg, így nagyobb érték esetén a jármű gyorsabban fog fordulni, míg kisebb értéknél lassabban.

- A forgási sebesség befolyásolja, milyen hatékonyan képes az autó szűk kanyarokat bevenni vagy gyorsan reagálni a kormányzási parancsokra.
3. *vel*: (velocity) Az aktuális sebesség
- Az aktuális sebesség mutatja meg a jármű valós időben mért sebességét, amely folyamatosan változik a gyorsulás, lassítás és fékezés hatására.
 - Ez a változó az autó irányításában is kulcsszerepet játszik, mivel a különböző tereptípusokhoz és akadályokhoz igazodva szabályozza az autó sebességét.
4. *angle*: A jármű aktuális forgási szöge
- Az *angle* változó a jármű aktuális orientációját tartja számon az északi irányhoz képest. Ezt az értéket fokban adjuk meg, és 0-360 közötti tartományban mozog.
 - Az *angle* meghatározza, hogy az autó milyen irányba halad az aktuális sebességgel, valamint, hogy milyen szögben fordul el egy adott kormányzási parancs hatására.
4. *acceleration*: A jármű gyorsulása
- Az *acceleration* változó meghatározza, hogy az autó milyen gyorsan képes elérni az aktuális vagy maximális sebességét.
 - Gyorsítás: Az *acceleration* értéke hozzáadódik az aktuális sebességhez (*vel*), így az autó a mozgási parancsoknak megfelelően felgyorsul.
 - Lassítás: Lassításkor vagy hátramenetben ez az érték kivonódik az aktuális sebességből, csökkentve annak értékét.
 - Biztonsági Paraméterek: Az *acceleration* biztosítja, hogy a gyorsulás mértéke összhangban legyen a pálya körülményeivel és a maximális sebességgel. Ha az autó terepre kerül, az érték csökken, hogy az autó ne legyen kezelhetetlen.
5. *center*: Az autó középpontja
- A *center* pozíció meghatározza az autó középpontját a grafikus ábrázoláson belül, az aktuális *x* és *y* koordináták alapján.
 - A középpont fontos referenciapont az autó forgatásához és az érzékelők elhelyezéséhez. Az autó forgási szöge és iránya mind a *center* pozícióhoz igazodva módosul, biztosítva, hogy az elfordulás mindig a jármű közepénél történjen.
 - A szenzorok kiindulópontját is a *center* érték határozza meg, biztosítva, hogy az érzékelők mindig az autó közepétől távolodva figyeljék a környezetet. Ez elengedhetetlen az akadályok pontos észleléséhez, mivel a szenzorok szöge az autó szögéhez képest kerül meghatározásra.

4.1.2. A jármű mozgatása

A jármű mozgásának és forgatásának pontos leírása elengedhetetlen a szimuláció viselkedésének megértéséhez. A mozgás és forgatás metódusai teszik lehetővé az autó navigálását a pályán, egyaránt kezelve az aktuális sebességet, gyorsulást és az irányváltást.

A `move()` metódus a 4.1-es kódban látható, a jármű aktuális pozíciójának frissítéséért felelős, és az aktuális sebesség és szög alapján számítja ki az autó új helyzetét. Lássuk lépésről lépésre, hogyan működik. A `move()` metódus először az autó forgási szögét (`self.angle`) radián értékre váltja át a Python beépített `math.radians` függvényének segítségével. A trigonometriai függvények, mint a szinusz és koszinusz, radián értékekkel dolgoznak, ezért elengedhetetlen a szög átváltása. A `self.angle` szög fokban van tárolva, így a `math.radians` függvény ezt konvertálja át. A metódus a koszinusz és szinusz függvények használatával kiszámítja a jármű függőleges és vízszintes irányban történő elmozdulását. A függőleges elmozdulást a koszinusz függvény segítségével számolja ki. A függőleges elmozdulás a szög koszinuszával arányos, hiszen ez a tengely az autó előre irányuló mozgását reprezentálja. A vízszintes elmozdulást a szinusz függvény határozza meg, amely az aktuális szög szinuszcímértékét veszi figyelembe. A szinusz érték határozza meg, mennyit tér el az autó oldalirányban. Mindkét elmozdulási értéket megszorozzuk az aktuális sebességgel (`self.vel`), így megkapjuk az elmozdulás valós értékét a sebesség alapján. A függőleges elmozdulást kivonja a jármű jelenlegi y koordinátájából, míg a vízszintes elmozdulást az x koordinátából. Ez azért történik, mert a PyGame koordináta-rendszerében az origó a bal felső sarokban van, tehát lefelé haladva az y növekszik, jobbra haladva az x növekszik. Ez a kivonás biztosítja, hogy az autó előre mozduljon el, és a forgási szöggel összhangban lépjen a következő pozícióba.

```
1 def move()  
2     radians = math.radians(self.angle)  
3     vertical = math.cos(radians) * self.vel  
4     horizontal = math.sin(radians) * self.vel  
5     self.y -= vertical  
6     self.x -= horizontal
```

4.1. kódrészlet: Az autó mozgását lehetővé tevő metódus

A `move_forward()` metódus 4.2-es kódrészlet, a sebességnöveléshez használt függvény, amely először az aktuális sebességet (`self.vel`) növeli a gyorsulás (`self.acceleration`) értékével, de az új sebesség nem haladhatja meg a maximális sebességet

(*self.max_vel*). A Pozíció változtatása az új sebesség kiszámítása után a *move()* függvényt hívja meg, amely az aktuális pozíciót frissíti a sebesség és az autó szögének megfelelően.

```
1 def move_forward(self):
2     self.vel = min(self.vel + self.acceleration, self.max_vel)
3     self.move()
```

4.2. kódrészlet: Az előre mozgást lehetővé tevő módszer

A *move_backward()* módszer 4.3-as kódrészlet, sebességcsökkentés. Ebben a módszerben a hátrameneti mozgás sebessége kerül beállításra. Az aktuális sebességet csökkenti a gyorsulási értéknek megfelelően, de korlátozza a maximális hátrameneti sebességet az előremeneti sebesség felére. Az előremeneti mozgáshoz hasonlóan itt is a *move* módszer végzi az aktuális pozíció frissítését a sebesség és az autó szögének alapján.

```
1 def move_backward(self):
2     self.vel = max(self.vel - self.acceleration, -self.max_vel / 2)
3     self.move()
```

4.3. kódrészlet: A lassítást hátramenetelt lehetővé tevő módszer

A *rotate()* módszer 4.4-es kódrészlet, vagyis a forgatás módszer két paramétert vesz fel: *left* és *right*, amelyek a balra vagy jobbra forgatás irányát jelölik. Ha a *left* igaz, a módszer hozzáadja a forgási sebességet (*self.rotation_vel*) az aktuális szöghöz, ha a *right*, akkor pedig kivonja belőle. Így a szög értéke pozitív vagy negatív irányba változik a forgási sebesség függvényében. A módszer figyel, hogy az *angle* értéke ne lépje túl a 0-360 fokok tartományt. Ha 360 foknál nagyobb, akkor csökkenti, ha pedig 0 fok alá megy, akkor növeli az értéket.

```
1 def rotate(self, left=False, right=False):
2     if left:
3         self.angle += self.rotation_vel
4     elif right:
5         self.angle -= self.rotation_vel
```

4.4. kódrészlet: Az autó forgását szabályzó módszer

A *collide()* módszer egy ütközésvizsgáló függvény, amely azt határozza meg, hogy az autó (egy maszkot használva) érintkezésbe kerül-e a megadott maszk felületével. Ez

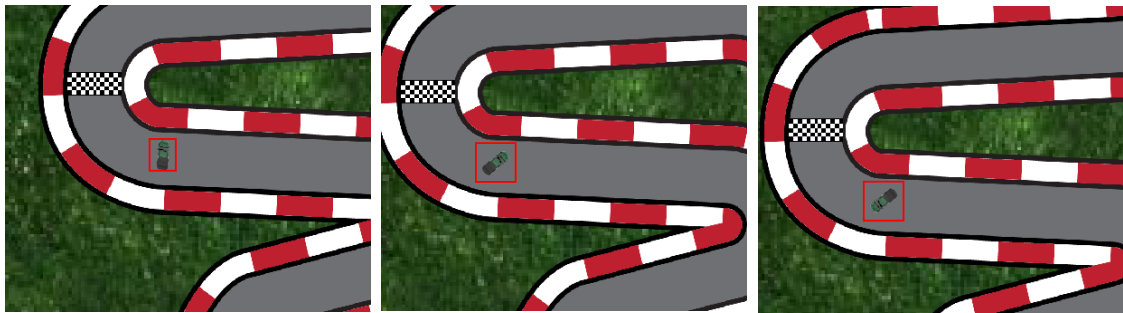
kulcsfontosságú az önvezető autó szimulációjában, mivel az autó megfelelő reagálását biztosítja, amikor például terepre vagy akadályra hajt. Az előző fejezetben volt szó a maszkolásról és a megadott példához hasonló a függvényünk. Az *offset* változó kiszámításával meghatározzuk a két maszk közötti eltolást. Az eltolást az autó (jelenlegi pozíció) és a másik felület (*x*, *y* pozíció) különbségeként számítjuk ki. Az eltolás azt jelzi, hogy a két maszk milyen pozícióban helyezkedik el egymáshoz képest. A *mask.overlap* metódus használatával ellenőrzi, hogy a két maszk átfed-e egymással az eltolás figyelembevételével. Az *overlap* függvény azt adja vissza, hogy a két maszk között van-e nem nulla metszet az eltolás alapján. Ha van átfedés, a metszet első pontjának koordinátáit adja vissza, egyébként *None* értéket.

```
1 def collide(self, mask, x=0, y=0)
2     offset = (int(self.x - x), int(self.y - y))
3     return mask.overlap(CAR_MASK, offset)
```

4.5. kódrészlet: Az ütközést szabályzó metódus

A *rotate_image_to_center()* [6/a] függvény a 4.6-os kódban látható és az autó grafikus megjelenítésének forgatásáért és pozicionálásáért felelős. Az autó aktuális szögének megfelelően forgatja el a képet, majd az elforgatott képet úgy pozicionálja, hogy annak középpontja egy meghatározott pont körül legyen. A függvény ezen kívül kirajzol egy piros színű keretet is, amely a forgatott kép méretét és helyzetét mutatja. A függvény első lépésként a *pygame.transform.rotate* függvénnyel elforgatja a képet az aktuális szög (fokban) alapján. Az eredeti képhez képest az elforgatott kép mérete változhat, ami azt jelenti, hogy az elforgatott kép doboza nagyobb vagy kisebb lehet az eredetihez képest. A *get_rect()* metódust hívja meg az elforgatott képen, hogy lekérje annak új méretét és keretét. A középpont újrapozicionálásához az eredeti kép keretén (*get_rect*) keresztül meghatározza az eredeti kép középpontját (*topleft* alapján). Az új, elforgatott képnek ezt a középpontot kell megőriznie, így az új keret (*new_img_rect*) középpontját is ennek megfelelően határozzuk meg. Az elforgatott képet a *win.blit* függvénnyel rajzolja ki a képernyőre, az új keret bal felső sarkánál (*new_img_rect.topleft*), így a kép helyzete és szöge is megfelelő lesz. A függvény az elforgatott képhez piros keretet rajzol a *pygame.draw.rect* függvénnyel. Ez a keret az elforgatott kép méretét és pozícióját mutatja, segítve ezzel a vizuális hibakeresést és a kép pontos elhelyezését a pályán. A *rotate_image_to_center* függvény gondoskodik az autó grafikus ábrázolásának helyes forgatásáról és pozicionálásáról. Az elforgatott kép

újrapozícionálása révén az autó középpontja mindig a megfelelő helyen marad, és pontosan leképezi az autó aktuális helyzetét és szögét a szimulált környezetben.



4.1. ábra: A jármű forgatása

```
1 def rotate_image_to_center(window, img, position, rotation_angle):
2     rotated_img = pygame.transform.rotate(img, rotation_angle)
3     new_img_rect = rotated_img.get_rect(center=img.get_rect(topleft=position).center)
4     window.blit(rotated_img, new_img_rect.topleft)
5     pygame.draw.rect(window, (255, 0, 0), new_img_rect, 2)
```

4.6. kódrészlet: Az autó grafikus forgatásáért felelős módszer

4.2. Szenzorok bevezetése

A szenzorok fontos elemei az autó környezetének észleléséhez, alább ennek implementációját tekinthetjük meg.

A `collect_sensor_data()` függvény részletesen bemutatja, hogyan gyűjti össze és dolgozza fel a szenzorokból származó adatokat egy autonóm jármű szimulációs környezetben. A függvény egy sor fontos lépést tartalmaz, amelyek lehetővé teszik a jármű környezetének pontos érzékelését és a megfigyelt információk alapján döntések meghozatalát, paraméterként az autó egy példányát kapja. A függvény kezdetén létrehozuk a `sensor_data` listát, amely kezdetben üres, viszont mindig a szenzorok aktuális számával inicializáljuk. A szög kiszámítása az `absolute_sensor_angle` változóval történik. A `car.angle` a jármű jelenlegi orientációját jelöli fokban. A `sensor_angle` az adott szenzor relatív szöge a járműhöz képest radiánban. Az eredményül kapott `absolute_sensor_angle` megadja a szenzor abszolút orientációját a PyGame koordináta-rendszerében. Miután megkaptuk az abszolút szögeket, a következő lépés a szenzorok végpontjainak koordinátáinak kiszámítása. Ez segít meghatározni, hogy a szenzorok milyen területeket látnak és hol vannak az akadályok. A szenzorok fizikai helyzete a járműhöz rögzített, azonban a jármű orientációja (azaz az, hogy melyik irányba néz) változhat

a mozgása során. Emiatt szükséges az, hogy a szenzorok pontos kezdőpontjait dinamikusan számítsuk ki, figyelembe véve a jármű aktuális orientációját. Ez teszi lehetővé, hogy a szenzorok által érzékelt adatok helyesen értelmezhetők legyenek a jármű aktuális helyzetéhez viszonyítva. A számítás kiindulópontja a jármű középpontjának x -koordinátája (`car.center[0]`), ami a jármű geometriai közepét jelzi az x tengelyen. A jármű szélességének felével (`CAR_WIDTH / 2`) szorozva a jármű aktuális irányának koszinuszát adja meg, hogy mennyire mozduljon el a szenzor a jármű középpontjától az x tengely mentén. A koszinusz függvényt használjuk, mert ez adja meg a vízszintes eltolódás mértékét az adott irányban. A jármű szögét (`car.angle`) radiánba konvertáljuk (`math.radians(car.angle)`), mivel a trigonometrikus függvények radiánban értelmezettek. Ez a konverzió biztosítja, hogy az orientáció helyesen számíthasson bele a helyzet meghatározásába. A `start_y` alapértéke, az y koordináta alapja a jármű középpontjának y -koordinátája (`car.center[1]`). Itt is a jármű szélességének felét veszi figyelembe, de a szinuszt használjuk, ami megadja a függőleges eltolódás mértékét az adott irányban. Az orientációt itt is figyelembe kell venni, így a jármű szöge szintén radiánba van konvertálva. Az érzékelő végeinek koordinátái az `end_x`, `end_y` azt a pontot jelölik, ahol a szenzor érzékelése véget ér, azaz meddig terjed az érzékelési zóna. A végpont meghatározása az alábbi lépéseken keresztül történik. Kiindulópontnak a `start_x` és `start_y` változókat használjuk, amelyek a szenzor kezdőpontjának x és y koordinátáit jelentik. A szenzor hossza `car.sensor_length` azt határozza meg, milyen távolra képes a szenzor érzékelni. Ezt a távolságot a szenzor hosszának koszinuszával szorozva számítjuk ki, ami az abszolút szenzorszög `absolute_sensor_angle` koszinuszát vagy az `end_y` esetén a szinuszt jelenti. Az előjel váltás szükséges, mert a koordináta-rendszerben az y tengely felfelé növekvő, tehát a negatív irány a felfelé haladást jelenti. A `found_obstacle` változó jelzi nekünk, ha akadályt talált a szenzor által érzékelt zónában. Az `obstacles_list`-en végigiterálva leellenőrizzük a szenzorok végpontjainak ütközését valamelyik akadállyal, és ha talált egyezést feltölti az adott index értékét a megfelelő szín betűjelével. Ha nem talált akadályt a ciklus megy tovább az úttípusokra és leellenőrzi azok ütközését a `check_sensor_collision()` metódussal aminek paraméterei az ellenőrizendő maszk és a szenzorok végpontjai. Összesen 5 színt különböztetünk meg, 2-öt az akadályoknak, 3-at pedig az úttípusoknak. Fekete, ha gyalogost érzékel, narancsot ha sarat, zöldet ha utat érzékel, citromot ha az út szélét, és pirosat ha a fűvet. Ha a `sensor_data` lista hossza nem egyezik meg a szenzorok számával (`NUM_SENSORS`), akkor `ValueError` kivételt dobunk, jelezve, hogy nem elegendő adat áll rendelkezésre. Amennyiben bármilyen kivétel keletkezik a feldolgozás során,

az logolásra kerül, és minden szenzor adatát pirosra állítjuk, elkerülve ezzel a pálya szélén bekövetkező hibákat a szenzorok részéről. [A teljes kód az 1sz_melleklet_sensor_collector.txt című mellékletben tekinthető meg.](#)

5. Gépi tanulási módszerek

5.1. Gépi tanulás

A következőkben a Gépi tanulásról lesz szó, mely kidolgozása az [1] alapján történik.

A gépi tanulás a mesterséges intelligencia egyik ága, amely algoritmusok és statisztikai modellek használatán keresztül képes a gépeknek olyan utasításokat adni, hogy feladatokat hajtsanak végre anélkül, hogy explicit programozásra lenne szükségük. A gépi tanulás lehetővé teszi számunkra, hogy olyan problémákkal, feladatokkal is megbirkózzunk, ami számunkra túl bonyolult és egy előre lefixált, ember által megírt program nem lenne képes. A számítógépek számára lehetővé teszi, hogy tapasztalatokból tanuljanak, mintákat és összefüggéseket ismerjenek fel, és döntéseket hozzanak minimalizált emberi beavatkozással. Ennek a célja, hogy olyan algoritmusokat fejlesszen, amelyek adatok elemzésével, minták és összefüggések felismerésével képesek legyenek javítani saját teljesítményüket a feladatok elvégzése során. A gépi tanulási módszerek alapja a statisztikai elemzés, ahol az adatokat modell segítségével elemezzük, hogy előrejelzéseket készítsünk vagy döntéseket hozzunk. Ezek az algoritmusok lehetővé teszik, hogy a gépek komplex feladatokat hajtsanak végre, például arc- és képfelismerést, szövegértést, vagy akár járművezetést anélkül, hogy pontos utasításokat kellene megfogalmazni a feladat minden egyes aspektusára.

A gépi tanulási modell felállítása egy többlépcsős folyamat, amely magában foglalja az adatgyűjtést, előkészítést, modell választást, tanítást, értékelést és végül a modell alkalmazását. Lássuk részletesen, hogyan áll fel egy gépi tanulási modell:

Adatgyűjtés: Minden gépi tanulási projekt az adatokkal kezdődik. Szükséges egy megfelelő adathalmaz gyűjtése, ami lehet magunk által kreált, vállalati belső adatok vagy akár az internetről gyűjtött adatok. Az adatoknak relevánsnak kell lenniük a megoldandó problémára, és elegendő mennyiségűnek kell lenniük ahhoz, hogy a modell hatékonyan tanulhasson belőlük.

Adatok előkészítése: Az adatelőkészítés a legidőigényesebb lépések egyike. Ez magában foglalja az adatok tisztítását, normálását, hiányzó értékek kezelését és esetleges kimeneti változók kódolását. Az adatokat gyakran szükséges transzformálni vagy normalizálni, hogy a gépi tanulási algoritmusok hatékonyabban dolgozhassanak vele. Ezen felül az adatokat feloszthatjuk tanító, validációs és teszt készletekre.

Modell kiválasztása: A probléma típusától (osztályozás, regresszió, klaszterezés stb.) függően kiválaszthatjuk a megfelelő gépi tanulási modellt. Lehet ez egy egyszerű lineáris

regressziós modell, döntési fa vagy bonyolultabb neurális hálózat. Fontos, hogy a modell jól illeszkedjen az adatokhoz és a problémához.

Tanítás: A modellt a tanító adatkészleten tanítjuk, ahol az algoritmus a bemeneti jellemzők és a kívánt kimenet közötti kapcsolatokat igyekszik megtanulni. Ebben a fázisban állítjuk be a modell hiperparamétereit is, mint például a tanulási ráta vagy a döntési fák mélysége. A tanítási folyamat során a modell súlyait iteratíván frissítjük a hiba minimalizálása érdekében.

Értékelés: Miután a modell a tanító adatokon megtanulta a feladatot, a teljesítményét a validációs és teszt adatkészleteken értékeljük. Itt mérjük a modell pontosságát, érzékenységét, specifikitását, stb. Ezen kívül vizsgálhatjuk az overfitting vagy underfitting jeleit is.

Finomhangolás és Optimalizálás: A modell értékelése után szükség lehet a modell finomhangolására, hiperparamétereinek újrahangolására, vagy akár a modell struktúrájának módosítására a teljesítmény javítása érdekében.

5.1.1. Gépi tanulás alkalmazásai

A **regresszió**, ahol a cél a bemeneti változók alapján numerikus értékek előrejelzése. Ez különösen hasznos olyan területeken, mint a pénzügyi előrejelzés vagy az ingatlanárak meghatározása. Az ilyen problémák megoldására a tanuló algoritmust arra kérjük, hogy adjon egy kimeneti függvényt $f: \mathbb{R}^n \rightarrow \mathbb{R}$.

A **transzkripció**, vagy optikai karakterfelismerés problémában a rendszernek egy viszonylag szerkezet nélküli adatábrázolást kell diszkrét szöveges formába átírnia. Például az optikai karakterfelismerés esetében a programnak egy szöveget tartalmazó képet kell karakterek sorozatává alakítania. A Google Street View ezt mélytanulási technikák segítségével végzi el címek feldolgozására. A beszéd felismerés egy másik példa, ahol a program hanghullámokat kap, és karakterek vagy szavak sorozatát állítja elő a hangfelvétel alapján.

A **gépi fordítás**, ahol a gépeknek nyelvi szimbólumsorozatokot kell egyik nyelvről a másikkra átfordítaniuk, szintén egy jelentős alkalmazási terület. Ebben az esetben a gépek képesek a szöveg értelmét is megérteni és azt úgy reprodukálni, hogy az az adott célnyelven értelmes és helyes legyen. Ez a technológia már számos nyelven elérhető és jelentősen megkönnyíti a különböző nyelvek közötti kommunikációt.

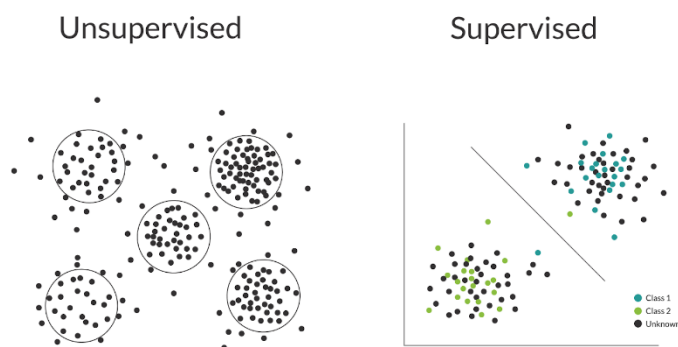
Az **anomália detektálásnál** a rendszernek ki kell szűrnie a nem oda illő, atipikus, vagy szokatlan eseményeket a normális tevékenységek közül. Ezt gyakran alkalmazzák pénzügyi csalások felderítésére, vagy a hálózati biztonság javítása érdekében.

5.1.2. Gépi tanulás kategóriái

A gépi tanulás három fő kategóriába sorolható: felügyelt tanulás, felügyelet nélküli tanulás és a megerősítéssel tanulás.

1. A felügyelt tanulás vagy supervised learning (5.1-es ábra) az látható során az algoritmusoknak címkézett adatokból kell tanulniuk, ami azt jelenti, hogy minden adatpont mellé hozzárendelünk egy címkét, amely alapján a modell megtanulhatja az adott kimenetel előrejelzését. Tipikus felhasználási területek közé tartozik a kép- és beszédfelismerés, valamint az e-mailek szűrése.

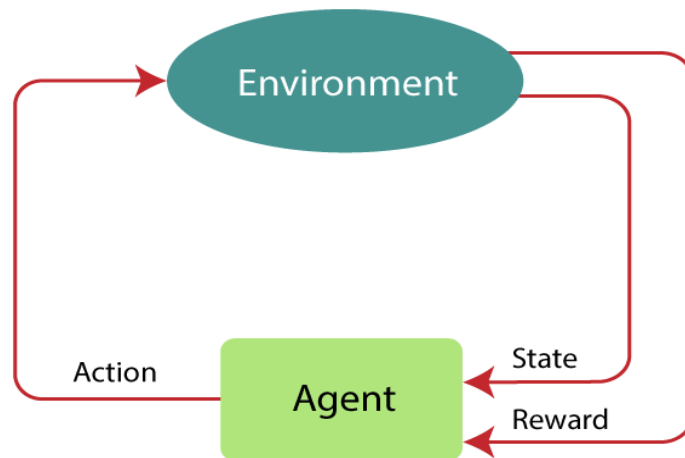
2. Ezzel szemben a felügyelet nélküli vagy unsupervised learning (5.1-es ábra) tanulás olyan adatokon dolgozik, amelyekhez nem tartoznak előre definiált címkék. Ebben az esetben az algoritmusok az adatstruktúrákban rejlő mintázatok, csoportok és összefüggések feltárására törekcszenek, mint például vásárlói adatok csoportosítása vásárlási szokások alapján. Ez a típusú tanulás segíthet új összefüggések és trendek felfedezésében, amelyek nem nyilvánvalóak az adatok pusztá vizsgálata során.



5.1. ábra [13]: Bal oldalon a felügyelet nélküli, csoportosított minták, jobb oldalon a felügyelt, címkézett

3. A megerősítéssel tanulás vagy reinforcement learning (5.2-es ábra) egy dinamikus környezetben történik, ahol a modellnek döntéseket kell hoznia, és ezekért jutalmat vagy büntetést kap. Az agent egy adott környezetben különböző cselekvéseket hajt végre egy már előre meghatározott cél érdekében. Az agent-et egy meghatározott mérőszám alapján, amelyek általában pontszámok, jutalmazást vagy büntetést kap a cselekvéséért. Ez ösztönzi az agent-et, hogy a jó gyakorlatot folytassa és eldobja a rosszakat. A folyamatos ismétléssel megtanulja a legjobb stratégiákat a probléma megoldásához. Ez a tanulási forma különösen

hasznos olyan alkalmazásokban, ahol a döntéseknek hosszú távú következményei vannak, mint például a videojátékokban vagy autonóm járművek vezérlésében.



5.2. ábra [14]: A megerősítéses tanulás ciklusa

5.1.3. Overfitting és Underfitting

A gépi tanulásban az overfitting - túltanulás és az underfitting – alultanulás (5.3-as ábra) két gyakori probléma, amely jelentősen befolyásolhatja a modellek teljesítményét. Ezek a problémák akkor fordulnak elő, amikor a modell nem képes általánosítani az új, még nem látott input adatokra az eredeti, tanító adatkészleten túl. Az overfitting és underfitting megértése kulcsfontosságú a hatékony gépi tanulási modellek kialakításához.

Az overfitting akkor történik, amikor egy gépi tanulási modell túlságosan jól illeszkedik a tanító adathalmazra, beleértve annak zaját és kivételeit is. Ebben az esetben a modell olyan összetetté válik, hogy képes az egyedi mintázatokat, még a véletlen zajokat is megtanulni, ami azonban a valós, általánosított használat során problémákat okozhat. A túltanulás gyakran akkor fordul elő, amikor a modell túl sok paraméterrel rendelkezik a rendelkezésre álló adatokhoz képest, vagy ha a tanulási folyamat túl hosszú ideig tart. Az overfitting jelei közé tartozik, hogy a tanítási halmazon a modell hibája alacsony, de a validációs vagy teszhalmazon magas.

Az overfitting elkerüléséhez rendelkezésre áll több stratégia is:

- Adatok bővítése: Az adathalmaz mesterséges növelése, például képek forgatásával vagy eltolásával, amely segít több tanulási példát biztosítani a modell számára.
- Korai leállás: A tanulási folyamat megállítása, amint a validációs hiba növekedni kezd, ami segít elkerülni a modell túltanulását.

- Regularizáció: Technikák, mint például L1 vagy L2 regularizáció, amelyek büntetik a modell nagy súlyait, és segítenek megakadályozni a túlzott illeszkedést.
- Dropout: Egy rétegben véletlenszerűen kihagy bizonyos neuronokat a tanítási fázisban, ezzel segítve a modell általánosító képességét.

Az underfitting az overfitting ellentéte. Akkor fordul elő, amikor a modell túl egyszerű, és nem képes megtanulni az adathalmaz alapvető struktúráit. Az underfitting gyakran abból adódik, hogy a modell nem rendelkezik elegendő paraméterrel vagy réteggel, hogy megfelelően leírja az adatok bonyolultságát, vagy ha a tanítási folyamat túl korán leáll. Az underfitting jelei közé tartozik, hogy a modell magas hibarátaival rendelkezik mind a tanítási, mind a validációs adathalmazon.

Az underfitting kezelésére a következő módszereket alkalmazhatjuk:

- Modell komplexitásának növelése: Több réteg vagy neuron hozzáadása a modellhez, hogy az képes legyen jobban megtanulni az adatok bonyolultságát.
- Több jellemző (feature) hozzáadása: Néha az underfitting oka az lehet, hogy a modell nem rendelkezik elegendő információval az adatokról. Ebben az esetben új jellemzők létrehozása vagy meglévők bővítése (feature engineering) segíthet a modellnek jobban megérteni az adatstruktúrát.
- Tanulási algoritmus paramétereinek finomhangolása: Néha az algoritmus paramétereinek, mint például a tanulási ráta vagy a regularizációs paraméterek módosítása segíthet a modellnek, hogy jobban illeszkedjen az adatokhoz anélkül, hogy a modell struktúráját alapvetően megváltoztatnánk.
- Több tanulási idő: Növelhetjük a tanítási iterációk számát vagy a tanulási folyamat időtartamát, hogy a modell több lehetőséget kapjon az adatokból való tanulásra. Ez különösen hasznos lehet, ha a modell tanulási görbéje azt mutatja, hogy a hiba folyamatosan csökken, de még nem stabilizálódott.



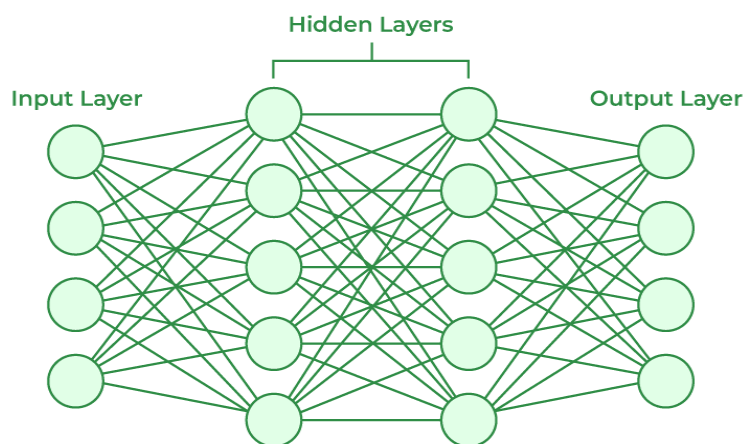
5.3. ábra [16]: alultanulás és túltanulás adatvizualizációja

5.2. Neurális hálózatok

A következő fejezetben a neurális hálózatokról lesz szó, mely kidolgozása a [1] és [7] alapján történt. A neurális hálózatok a gépi tanulás eszközei, amelyek az emberi agy neuronjainak működését modellezzik digitális formában. Ezen modellek célja az, hogy képesek legyenek tanulni és mintákat felismerni komplex adatokból, amelyek más hagyományos algoritmusok számára nehezen értelmezhetőek. Neurális hálózatokat széles körben alkalmaznak a képfeldolgozástól kezdve a szövegértelmezésen át az összetett játékstratégiák kialakításáig.

A neurális hálózatok alapvető építőeleme a neuron, amely egy számítási egység. Egy hálózat több rétegből állhat, amelyeket bemeneti, rejtett és kimeneti rétegeknek nevezünk. A bemeneti réteg fogadja az adatokat, a rejtett rétegeken keresztül történik a feldolgozás, és a kimeneti réteg adja meg a hálózat válaszát, ahogy az 5.4-es ábra is mutatja:

- **Bemeneti Réteg:** Az adatokat fogadja, amelyek lehetnek képpontok egy képből, szavak vagy karakterek egy szövegből, vagy más típusú mérési adatok.
- **Rejtett Rétegek:** A bemeneti rétegből érkező információkat feldolgozó rétegek, amelyek a hálózat tanulásának és gondolkodásának helyszínei. A rejtett rétegek száma és mérete határozza meg a hálózat komplexitását és képességét.
- **Kimeneti Réteg:** A hálózat által adott választ produkálja, amely lehet egy osztály címke, egy valószínűségi érték, egy folytonos számérték vagy akár egy komplex döntési struktúra is.



5.4. ábra [17]: Neurális hálózat architektúrája

A neurális hálózatok működése az aktivációs függvényen, a súlyokon és a bias értékeken alapul. Minden neuron kimenete az aktivációs függvény alkalmazásával számítható ki, amely egy matematikai képlet, például a szigmoid, tanh vagy ReLU függvény. A neuronok bemeneteinek súlyozása és a bias hozzáadása után az aktivációs függvény határozza meg a neuron kimenetét.

5.2.1. Bias

A neurális hálózatokban a bias egy kritikus komponens, amely a neuronok működését alapvetően befolyásolja. A bias egy olyan további bemenet a neuronokban, amelyet általában az aktivációs függvény előtt adnak hozzá a súlyozott bemenetek összegéhez. Ez a konstans érték lehetővé teszi, hogy a neuronok kimeneti szintjét finomhangoljuk, növelve ezzel a hálózat modellalkotási képességét és általánosíthatóságát.

A bias egy olyan paraméter a neurális hálózatokban, amely lehetővé teszi a modell számára, hogy jobban illeszkedjen a különböző adathalmazokhoz azáltal, hogy a lineáris kimeneti összegzéshez hozzáad egy konstans értéket. A bias fontos szerepet játszik a neurális hálózatok rugalmasságának növelésében, mivel segít korrigálni azokat az eseteket, amikor az összes bemenet nulla (vagy közel nulla) értéket vesz fel. Egy bias nélküli modell esetében, ha minden bemenet nulla, a kimenet is nulla lenne, függetlenül a súlyoktól. A bias hozzáadásával a neuron aktiválható akkor is, ha a bemeneti jellemzők nem hordoznak különösebb információt, így a hálózat képes általánosítani és jobban kezelni az adatok kisebb variációit.

A bias azért különösen fontos, mert lehetővé teszi, hogy az aktivációs függvény elmozduljon annak érdekében, hogy jobban illeszkedjen a kívánt kimenethez. Például a

szigmoid és tanh aktivációs függvények esetében a bias befolyásolja, hogy az aktivációs függvény görbéje milyen kimeneti értékek körül koncentrálódik, ami segíthet a tanulási folyamat során elérni a kívánt aktivációs mintákat.

A tanulási folyamat során a biasokat is frissíteni kell, hasonlóan a súlyokhoz. A back-propagation algoritmus segítségével a biasokat úgy frissítik, hogy azok csökkentsék a kimeneti hibát. A bias frissítési szabálya a következő:

$$b = b - \eta \frac{\partial E}{\partial b}$$

5.1-es képlet: bias frissítés

ahol η a tanulási ráta, $\frac{\partial E}{\partial b}$ pedig a hibafüggvény bias szerinti parciális deriváltja.

5.2.2. Aktivációs függvények

Az aktivációs függvények fontos szerepet játszanak a neurális hálózatokban, mivel ezek határozzák meg, hogy egy adott neuron aktiválódjon-e, és ha igen, milyen intenzitással. Ezek a függvények nemlineáris tulajdonságaik révén teszik lehetővé, hogy a neurális hálózatok összetett és nemlineáris mintákat is képesek legyenek tanulni és modellezni. Néhány gyakran használt aktivációs függvény:

1. A szigmoid aktivációs függvény egy klasszikus választás, amely 0 és 1 közötti értékeket ad vissza, így ideális bináris osztályozási feladatokhoz. Azonban ritkán használják a rejtett rétegekben a gradiens eltűnés problémája miatt.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

5.2. képlet: szigmoid aktivációs függvény

2. A ReLU egy népszerű aktivációs függvény, amelyet gyakran használnak mély neurális hálózatokban, mivel segít kezelni a gradiens eltűnés problémáját, és gyorsítja a konvergenciát. A ReLU függvény nullát ad vissza negatív bemenetek esetén, és lineárisan növekszik pozitív bemeneteknél.

$$f(x) = \max(0, x)$$

5.3. képlet: ReLU aktivációs függvény

3. A tanh függvény hasonló a szigmoidhoz, de -1 és 1 közötti értékeket ad vissza, ami jobb általánosítást tesz lehetővé bizonyos esetekben. Mint a szigmoid, a tanh is szenvedhet a gradiens eltűnés problémájától mély hálózatokban.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

5.4. képlet: Hiperbolikus tangens

4. A softmax függvény gyakran használatos a kimeneti rétegben többosztályos osztályozási feladatoknál, mivel a kimenetet valószínűségi eloszlássá alakítja, ahol minden osztály valószínűsége 0 és 1 közötti, és az összes valószínűség összege 1.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

5.5. képlet: Softmax függvény

Az aktivációs függvények megválasztása nagymértékben függ a specifikus feladattól és a hálózat architektúrájától, és döntően befolyásolhatják a modell tanulási dinamikáját és teljesítményét.

5.2.3. Veszteségfüggvények

Az MSE, vagyis a négyzetes hibaátlag, egy népszerű hibamérték a gépi tanulásban, különösen regressziós problémák esetén. Ez az érték az előrejelzések pontosságának mérésére szolgál, összehasonlítva a modell által előállított kimeneteket a tényleges adatokkal. Az MSE segítségével értékelhető, hogy egy adott modell mennyire jól illeszkedik az adathalmazhoz.

Az MSE kiszámítása az előrejelzett értékek (modell kimenetek) és a valós értékek (célváltozók) közötti eltérések négyzetének átlagaként történik. Formálisan:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

5.6. képlet: Mean Squared Error függvény

ahol: n az elemek száma a halmazban, y_i az i -edik valós érték, $\hat{y}_i$ az i -edik valós előrejelzett érték. Az MSE különösen érzékeny a nagy hibákra, mivel a különbségeket négyzetre emeli. Ez azt jelenti, hogy a nagy eltérések nagymértékben befolyásolják az összesített hibaértéket, ami

különösen hasznos lehet azokban az alkalmazásokban, ahol a nagy hibák nagyobb problémát jelentenek. Az MSE értéke mindig nemnegatív, és akkor éri el a minimális, azaz 0 értéket, amikor minden előrejelzés tökéletesen egyezik a valós értékekkel. Az MSE folytonos és differenciálható, ami előnyös a gradiens-alapú optimalizációs algoritmusokban, mint például a gradiens csökkenés, melyek a hibafüggvény gradiensét használják a modell paramétereinek (pl. súlyok) frissítésére. Az MSE a regressziós modellek alapértelmezett hibamértéke, ahol a cél a valós számokból álló kimeneti értékek pontos előrejelzése.

A Categorical Crossentropy veszteségfüggvényt tipikusan osztályozási feladatokban használnak, ahol a kimeneti osztályok egymást kizáró kategóriák (pl. macska, kutya, madár). Ez a függvény méri, hogy a modell által előrejelzett valószínűségi eloszlás mennyire tér el a tényleges címkék által képviselt eloszlástól. A kategorikus keresztentrópia különösen hasznos mélytanulási modellekben, ahol a kimeneti réteg aktivációs függvénye a softmax, ami a kimeneti réteg neurális hálózatának kimeneteit valószínűségi értékekké alakítja. A CC képlete formálisan:

$$H(p, q) = - \sum_i p_i \log(q_i)$$

5.7. képlet: Kategorikus keresztentrópi függvény

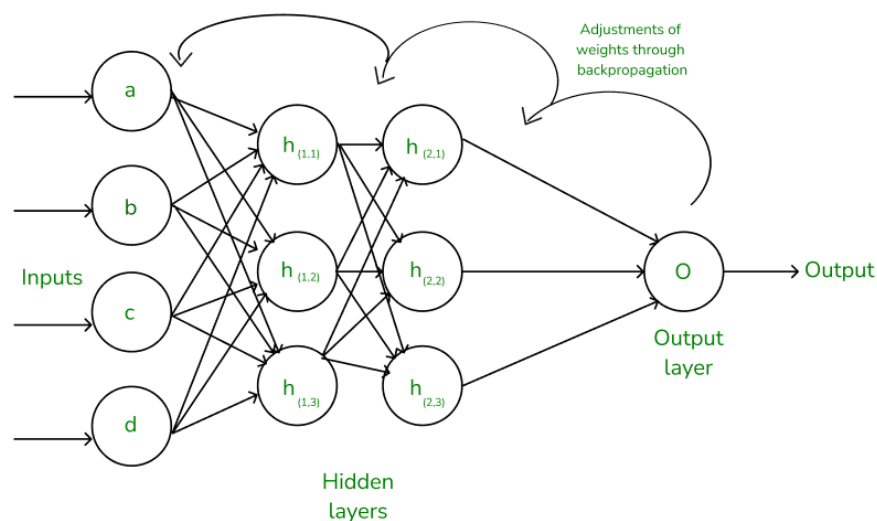
ahol: p a tényleges címkék valószínűségi eloszlása (one-hot encoded formában), q a modell által előrejelzett valószínűségek eloszlása, p_i és q_i az i -edik osztályra vonatkozó tényleges, illetve előrejelzett valószínűség, A logaritmus alapja természetes logaritmus (e alapú). A kategorikus keresztentrópia segít abban, hogy a modell hogyan tanuljon meg pontosan osztályozni, optimalizálva a valószínűségi kimeneteket a tényleges címkéknek megfelelően. Minél kisebb a keresztentrópia értéke, annál közelebb van a modell által adott előrejelzés a valósághoz. Egy alacsony keresztentrópia érték azt jelzi, hogy a modell jó teljesítményt nyújt, míg egy magas érték azt sugallja, hogy a modell nem képes jól osztályozni az adott adathalmazon.

5.2.4. Feed-forward féle neurális hálózat

A feed-forward féle neurális hálózat [7] amelyet multilayer perceptrons-nak is nevezünk, ehhez egy népszerű felügyelt tanulási módszert alkalmaznak, az error back-propagation algoritmust. Az általános back-propagation (SBP) algoritmusban az FNN tanulása két lépésből áll: egy forward pass és egy backward pass fázisból. Az forward pass fázisban a bemeneti jel a hálózaton

áthalad előre felé, rétegről rétegre, ahol a súlyok rögzítettek; a backward pass fázisban a hiba visszaterjed, és a súlyokat egy hibajavítási szabály alapján állítjuk be. Számos esetben sikeresen alkalmazták, azonban ismert két jelentős hiányossága: a lassú tanulási sebessége és a paraméterekre való érzékenysége. Egy egyszerűbb probléma esetén is számos iterációra szükséges a kisebb hálózatok betanításához. Ilyen hátrányok mellett a tanulási ráta kulcsszerepet játszik a teljesítmény befolyásolásában, ezért gondosan kell azt megválasztani. Ha túl nagy a tanulási ráta a hálózat kaotikus viselkedést mutathat, ami megnehezíti a tanulás sikerességét. Ha viszont túl kicsi az lassú konvergenciához vezet, ami szintén nem kívánatos. Ezért elmondható, hogy az optimális tanulási ráta változik a tanítások során és a probléma kiküszöbölésére egy dinamikusan optimális tanulási ráta kidolgozása lenne a cél.

A back-propagation (5.5-ös ábra) egy olyan algoritmus, amely lehetővé teszi a neurális hálózat súlyainak hatékony frissítését a tanító adatkészleten keresztül, minimalizálva a hibát a kívánt kimenet és a hálózat által előállított kimenet között. Működése két fő lépésből áll, az előterjesztésből (forward pass) és a visszaterjesztésből (backward pass).



5.5. ábra [15]: back-propagation alapú neurális hálózat működése

1. Az előterjesztés lépésben az adatok a hálózati bemeneti rétegekbe kerülnek, majd rétegenként haladnak előre, minden neuron kimenetét kiszámítva egészen a hálózat kimenetéig. A neuronok kimenete kiszámítható ezzel a képlettel:

$$y = \sigma\left(\sum_{i=1}^n w_i x_i + b\right)$$

5.8. képlet: Előterjesztés

Itt x_i az adott neuron bemenetei, w_i azokhoz tartozó súlyok, b a bias, és σ az aktivációs függvény.

2. A visszaterjesztés során a hálózat által előállított kimenet és a kívánt kimenet (címke) közötti hiba alapján frissítjük a súlyokat. A hibát általában egy hibafüggvénnyel, mint például a négyzetes hibafüggvénnyel mérjük:

$$E = \frac{1}{2} \sum (t - y)^2$$

5.9. képlet: Visszaterjesztés

Itt t a célkimenet, y a hálózat által előállított kimenet. A hiba gradiensét a hálózat minden súlyára kiszámítjuk, hogy meghatározzuk, hogyan kell változtatni a súlyokon a hiba csökkentése érdekében:

$$\frac{\partial E}{\partial w_i} = (y - t) * y * (1 - y) * x_i$$

5.10. képlet: Hibafüggvény

Ahol $\frac{\partial E}{\partial w_i}$ a hibafüggvény gradiense a w_i súly szerint, és x_i a hozzá tartozó bemenet. A súlyokat ezután frissítjük a következőképpen:

A back-propagation algoritmus iteratív, vagyis a fenti lépéseket addig ismételjük, míg a hálózat teljesítménye megfelelően nem javul, vagy a tanulási folyamat egy előre meghatározott kritérium szerint véget nem ér. Az iterációk során a hálózat egyre jobban megtanulja az adatokban rejlő összefüggéseket, és egyre pontosabban képes előrejelezni vagy osztályozni az új adatokat.

5.3. Q-tábla

Az alábbi fejezet a [8] alapján lett kidolgozva.

A Q-tábla a Q-learning algoritmus központi eleme, amely egy megerősítéses tanulási módszer. A Q-learning egy egyszerű, de hatékony algoritmus, amely az állapotok és cselekvések közötti optimális döntéshozatalt tanulja meg. A Q-tábla segítségével az agent képes megtalálni a legjobb cselekvést minden állapotban, hogy maximalizálja a hosszú távú jutalmát.

5.3.1. A Q tábla alapjai

A Q-tábla a Q-learning algoritmus központi eleme, amely egy megerősítéses tanulási módszer. A Q-tábla egy táblázat, amely minden lehetséges állapot-cselekvés párhoz egy Q-értéket rendel. Ez a Q-érték azt mutatja, hogy az adott állapotban az adott cselekvés milyen hosszú távú jutalmat eredményezhet. Az állapotok az adott probléma összes lehetséges konfigurációját képviselik, míg a cselekvések azok a lépések, amelyeket az agent végrehajthat. A Q-tábla kezdetben véletlenszerű értékeket tartalmaz, és az algoritmus során folyamatosan frissül a tapasztalatok alapján.

5.3.2. Markov döntési folyamatok

A Q-learning algoritmus és a Q-tábla megértéséhez alapvető fontosságú a Markov döntési folyamatok (MDP-k) fogalma. Az MDP-k matematikai keretrendszert biztosítanak a döntéshozatal és az optimalizálás problémáinak modellezésére. Egy MDP formálisan egy öttagú halmazzal írható le (S, A, P, R, γ) :

- S (State space): Az összes lehetséges állapot halmaza, amelyben a rendszer lehet. Például egy rácson mozgó robot esetén az állapot lehet a rács egy adott cellája.
- A (Action space): Az összes lehetséges cselekvés halmaza, amelyeket az agent végrehajthat. Például egy autó irányításánál lehetséges cselekvések az előre, hátra, jobb, balra mozgás
- P (Transition probability): Az átmeneti valószínűségek, amelyek meghatározzák, hogy egy adott állapotban végrehajtott adott cselekvés milyen valószínűséggel vezet egy másik állapotba. Formálisan $P(s'|s, a)$, ahol s az aktuális állapot, a a cselekvés, és s' a következő állapot.
- R (Reward function): A jutalomfüggvény, amely megadja az azonnali jutalmat, amelyet az agent kap, amikor egy adott cselekvést hajt végre egy adott állapotban. Formálisan $R(s, a)$.
- γ (Discount factor): A diszkontfaktor egy valós szám $0 \leq \gamma \leq 1$, amely meghatározza a jövőbeli jutalmak jelenbeli értékét. Minél közelebb van a γ az 1-hez, annál inkább figyelembe vesszük a hosszú távú jutalmakat.

Az MDP-k célja, hogy meghatározzák az optimális policy-t, amely egy olyan függvény, amely minden állapothoz egy cselekvést rendel. Az optimális policy azt jelenti, hogy az agent az adott politikát követve maximális hosszú távú összesített jutalmat kap.

Az optimális policy meghatározása érdekében az alábbi lépéseket kell végrehajtani:

1. Az értékfüggvény $V(s)$ az adott állapotban várható hosszú távú összesített jutalmat adja meg. Formálisan:

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$$

5.11. képlet: Értékfüggvény

2. A Q-értékfüggvény $Q(s, a)$ az adott állapotban egy adott cselekvés végrehajtása esetén várható hosszú távú összesített jutalmat adja meg. Formálisan:

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$$

5.12. képlet: Q-értékfüggvény

3. Az értékfüggvény és a Q-értékfüggvény a Bellman-egyenletek segítségével iteratívan frissíthetők. Az értékiterációs módszerrel az értékfüggvényt addig frissítjük, amíg az konvergál.
4. A policy iteráció során két lépésben váltakozunk: policy értékelés és policy javítás. A értékelés során az aktuális policy alapján kiszámítjuk az értékfüggvényt, míg a javítás során az aktuális policy-t úgy módosítjuk, hogy az maximális jutalmat eredményezzen.

5.3.3. A Q-tábla frissítési szabálya

A Q-tábla frissítése a Bellman-egyenlet alapján történik. Az egyenlet segítségével az agent minden egyes lépés után frissíti a Q-értékeket a következőképpen [8]:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

5.13. képlet: Bellman egyenlet

ahol:

- $Q(s, a)$ az aktuális állapot és cselekvés Q-értéke,
- α a tanulási ráta, amely meghatározza, hogy mennyire gyorsan alkalmazkodik az új információkhoz,
- r a kapott jutalom,
- γ a diszkontfaktor, amely meghatározza a jövőbeni jutalmak súlyát,

- $\max_{a'} Q(s', a')$ a következő állapotban várható maximális Q-érték.

A Q-learning algoritmus hatékonysága nagyban függ az állapotok és cselekvések diszkrét természetétől. A Q-tábla mérete exponenciálisan nő az állapotok és cselekvések számának növekedésével, ezért a módszer leginkább kisebb, diszkrét állapotterekre alkalmazható. Az állapottereket gyakran egyszerűsítik vagy diszkrétizálják a gyakorlati alkalmazások során, hogy kezelhető méretű Q-táblát kapjanak.

6. SL és RL módszerek elkészítése

Ebben a fejezetben részletesen bemutatásra kerülnek a felügyelt és a megerősítéses tanulási módszerek előkészítése és elkészítése. Az SL módszer egy back-propagation alapú FNN segítségével tanulja meg a bemeneti adatok és a kimeneti célok közötti kapcsolatot, míg az RL módszer a Q-learning algoritmust használja, amely az agent interakciói alapján tanulja meg, hogyan érheti el a maximális jutalmat. Az alábbiakban bemutatom mindkét módszer létrehozásának lépéseit.

6.1. SL modell: FNN

Az alábbiakban kiderül, hogyan hozhatunk létre egy feed-forward neurális hálózatot a felügyelt tanulás módszerével. Az SL modell célja, hogy a bemeneti adatok alapján megtanulja a kimeneti célokat, például az önvezető autók szenzoradatai alapján a jármű sebességét és irányát.

6.1.1. Adatgyűjtés

Az implementált szimulációs környezet lehetővé teszi, hogy részletes adatokat gyűjtsünk egy autonóm jármű működéséről és környezetének reakcióiról. Az adatgyűjtési folyamat fontos lépés a gépi tanulási modellek későbbi tanításához, mivel az adatok minősége és relevanciája közvetlenül befolyásolja a tanulási eredményeket.

Az adatgyűjtés egy automatizál rendszerrel történt. Az `apply_control()` funkció elemzése alapján megfigyelhetjük, hogy ez a metódus számos bemeneti szenzoradatot és feltételt kezel, hogy megfelelően irányítsa az autót a környezetben. A függvény minden egyes meghívása során a következő lépéseken megy keresztül: A funkció kezdete a szenzoradatok összesítésével történik. Minden szenzoradat típus ('G', 'Y', 'R', 'B', 'O') darabszámát kiszámítja, amely alapján a környezeti viszonyokra reagál. Ha gyalogost érzékel ('B'), a funkció a járművet az észlelt gyalogos irányának ellentétes oldalára irányítja, lassítás mellett. Hasonlóan a gyalogoshoz, ha sarat érzékel ('O'), lassít és ellentétes irányba fordul. A piros, vagyis tiltott terület észlelésekor a funkció igyekszik elkerülni ezt a területet. Ha az észlelt piros zóna domináns egy oldalon, akkor a jármű az ellenkező irányba kanyarodik. Sárga, vagyis figyelmeztető jelzés esetén óvatosabb megközelítést választ. Ha kétoldalt sárgát érzékel, a jármű előre halad. A teljes kód megtekinthető az 'automated_car.py' fájlban.

A `save_drive_data()` függvény a szenzoradatok és a jármű sebességének mentésére szolgál egy CSV fájlba. A függvény működését részletesen elemezve láthatjuk, hogy hogyan

valósul meg az adatgyűjtés és tárolás. A függvény három paramétert fogad: `sensor_data` egy lista, amely a jármű különböző szenzorainak adatait tartalmazza, `velocity` a jármű aktuális sebességének értéke, `filename` az a fájlnev, ahova az adatokat menteni kívánjuk, alapértelmezett értékkel ellátva. Az `os.path.exists` függvény segítségével a kód ellenőrzi, hogy a megadott fájlnévvel rendelkező fájl már létezik-e a rendszerben. Ennek eredményét a `file_exists` változó tárolja. A fájlt `append` módban nyitjuk meg, ami lehetővé teszi új adatok hozzáadását a meglévő fájlhoz új sorokban. A `newline=''` paraméterrel biztosítjuk, hogy a sorvégek kezelése megfelelően történjen különböző operációs rendszereken. A `csv.writer()` függvénnyel adatot írhatunk a fájlba. Első adat írásakor létrehoz a fájlban egy fejléct, ami megadja az oszlopok nevét a strukturált, könnyebb értelmezés érdekében. A `data` változóban fűzzük össze a rögzítendő adatokat, amelyeket hozzáadunk a fájlhoz. [A teljes kód az 1sz_melleklet_sensor_collector.txt mellékletben tekinthető meg.](#)

6.1.2. Az adatok előkészítése

A modell kódja teljes terjedelemben megtekinthető a `model.py` fájlban.

Az adatok előkészítése kritikus lépés a felügyelt tanulási modell fejlesztésében. A nyers adatok gyakran zajosak, hiányosak vagy különböző skálákon vannak, ami negatívan befolyásolhatja a modell teljesítményét. Ezért az adatok előkészítése magában foglalja a tisztítást, normalizálást, átalakítást, a tréning- és tesztalmozokra való felosztást.

Az adattisztítás célja, hogy eltávolítsa vagy korrigálja a hibás, hiányos vagy irreleváns adatokat. Az adatbázisok gyakran tartalmaznak hiányzó értékeket, amelyeket pótolni vagy eltávolítani kell. A hiányzó értékeket lehetőség szerint pótolhatjuk a soron következő vagy átlagos értékekkel, vagy egyszerűen eltávolíthatjuk azokat a sorokat. A zajos adatok, amelyek nem relevánsak vagy hibásak, szintén eltávolításra kerülnek. Például, ha egy szenzor hibás adatokat szolgáltatott, azokat ki kell szűrni. Biztosítani kell, hogy az adatok konzisztens formátumban legyenek, például az időbélyegek egységes formátumúak legyenek, és az összes mértékegység ugyanaz legyen.

A `load_data()` függvény az adatok előkészítését végzi. Ennek során betölti, előfeldolgozza és transzformálja a szenzorokból származó adatokat, valamint a jármű sebességadatait egy olyan formátumba, amely alkalmas a tanulási algoritmusok alkalmazására. A függvény a Pandas könyvtárat használja a '`automated_driving_data_full.csv`' nevű fájl beolvasására. Ez a fájl tartalmazza a különböző szenzorok által gyűjtött adatokat és a jármű sebességadatait. A `pd.read_csv()` függvény segítségével ezek az adatok egy `DataFrame`

objektumba töltődnek be. A szenzorok által érzékelt adatok (például 'G' a pálya, 'Y' a pálya széle, 'R' a fű, 'B' gyalogos és 'O' sár jelölésére) numerikus kódokká alakulnak át. Ez a lépés szükséges ahhoz, hogy a gépi tanulási modellek, amelyek általában numerikus adatokkal dolgoznak, hatékonyan feldolgozhassák az információkat. A `replace` metódus használatával minden egyes kategorikus jelölés megfelelő numerikus értékre cserélődik. A `DataFrame`-ből kiválogatjuk azokat az oszlopokat, amelyek a szenzorok adatait tartalmazzák. Ezeket az adatokat egy `NumPy` tömbbe helyezzük az `values` attribútum segítségével, így kapjuk meg az X változót, ami a bemeneti adatokat jelenti. A `MinMaxScaler` a `scikit-learn` könyvtárból kerül felhasználásra az adatok skálázásához. A skálázás során az adatokat úgy transzformáljuk, hogy azok értékei 0 és 1 közé essenek. Ez elősegíti a tanulási folyamat stabilitását és segít a konvergencia gyorsításában, mivel az egyes jellemzők azonos skálán lesznek. Ezek voltak a bemeneti adatok, most kezeljük a kimenetet. A `DataFrame` 'velocity' oszlopából származó sebességértékek is tömbbé alakításra kerülnek, így kapjuk meg az `y_velocity` tömböt, ami a kimeneti céladatokat tartalmazza. Az X és y változókba betöltésre kerülnek az előkészített bemeneti és kimeneti adatok a `load_data()` függvény segítségével. Ezek az adatok szolgálnak az alapjául a modell tanításának és tesztelésének.

A `train_test_split` funkció a `scikit-learn` könyvtárból kerül felhasználásra az adatok felosztásához tanítási (x_{train} , y_{train}) és tesztelési (x_{test} , y_{test}) halmazokra. Az adatok felosztásánál a teszthalmaz mérete 20% (`test_size=0.2`), ami azt jelenti, hogy a modell 80%-on tanul és 20%-on teszti. A `random_state` paraméter biztosítja az adatok felosztásának reprodukálhatóságát.

A kimeneti adatok (y), amelyek a sebességértékeket tartalmazzák, transzponálásra kerülnek, hogy megfeleljenek a `train_test_split` követelményeinek. Ez a lépés azért szükséges, mert az y eredetileg egyetlen listaként van tárolva, a transzponálás során pedig a különböző sebességértékek különálló elemekké válnak a tömbben.

A y_{train} és y_{test} tömböket újraformázzuk, hogy minden egyes sebességérték külön tömbben legyen tárolva. Ez a lépés azért fontos, mert a neurális hálózatok gyakran igénylik, hogy a kimeneti változók különálló tömbökben legyenek kezelve a tanítási folyamat során.

6.1.3. Az FNN felépítése

A `create_model()` [2] függvény egy MLP típusú neurális hálózatot definiál, amely különböző rétegeket és konfigurációkat használ a jármű vezérléséhez szükséges döntéshozatali

modellezéséhez. Alább részletesen ismertetem a modell architektúráját, rétegeit, aktivációs függvényeit és egyéb fontos aspektusait.

A bemeneti réteg fogadja az előfeldolgozott és normalizált szenzor adatokat, amelyek az autó környezetéről származnak. Az `input_dim` paraméter határozza meg a bemeneti jellemzők számát, ami közvetlenül kapcsolódik a szenzorok által szolgáltatott adatok típusaihoz és számához.

A `Dense` a rétegek alkotják a modell tanuló részét, ahol minden neuron kapcsolatban áll az előző réteg minden neuronjával. A neuronok száma és az aktivációs függvények kiválasztása kritikus, mivel ezek határozzák meg a hálózat tanulási képességét és a nemlineáris problémák kezelésére való képességét.

A `dropout` rétegek a túltanulás elleni küzdelem egyik eszköze. Ezek véletlenszerűen kikapcsolnak egy adott százalékát a neuronoknak a tanulási fázis során, ezzel növelve a modell általánosítási képességét és csökkentve a túlilleszkedés esélyét.

Az egyik kimenet a sebesség értéke lesz. Egyetlen neuron alkotja, amely a jármű sebességét becsli meg. A veszteségfüggvényként használt mean squared error ideális folytonos értékek becslésére, mivel az MSE célja a tényleges és a becsült értékek közötti eltérés négyzetének minimalizálása.

Az irány kimenet öt neuronból áll, a softmax aktivációs függvény segítségével, ami a különböző irányok valószínűségét modellezi. A `categorical_crossentropy` veszteségfüggvény jó választás kategorikus kimenetekhez, mivel a keresztentrópia méri az eloszlások vagy valószínűségi predikciók közötti eltérést, így hatékonyan képezve a modellt a valós címkék és a predikciók közötti információelméleti távolság minimalizálására.

6.2. RL módszer: Q-learning

Az alábbiakban kiderül hogyan implementálhatjuk [9] az RL módszert egy Q-learning algoritmuson keresztül, amely döntéshozatali problémákra kiváló.

A `QLearningAgent` osztály inicializálásakor megadott paraméterek magukban foglalják az akciók számát (`action_size`), az állapotok számát (`state_size`), a tanulási rátát (`learning_rate`), a diszkontrátát (`discount_factor`), kezdeti felfedezési rátát (`initial_exploration_rate`), a felfedezés csökkenési rátáját (`exploration_decay`), és a minimális felfedezési rátát (`min_exploration_rate`). Inicializál egy Q-táblát, amely az állapotok és akciók várható jutalmait tárolja, kezdetben minden érték nullára van állítva

A `decide_action()` függvény az agent aktuális állapotát veszi alapul, és meghatározza a következő lépést. Az algoritmus az ϵ -greedy stratégiát használja, ami egyensúlyt teremt az exploration és az exploitation között. Ha egy véletlenszerűen generált szám kisebb, mint az aktuális `exploration_rate`, az agent véletlenszerű cselekvést hajt végre. Ez lehetővé teszi új stratégiák és cselekvések kipróbálását, amelyekről még nem gyűjtött elegendő információt. Ha a véletlen szám nagyobb, az agent a Q-tábla alapján választja ki a legjobb cselekvést, ami maximális jutalmat ígér az adott állapotban.

Az `update_policy()` függvény a tanulás szívének képezi, ahol a Q-értékek frissítése történik a Bellman-egyenlet alapján. Az `old_value` a Q-táblában tárolt jelenlegi érték az adott állapot-cselekvés párhoz. A `future_q` a következő állapotban elérhető maximális Q-érték, ami segít a jövőbeli döntések becslésében. A `new_value` kiszámítása a tanulási ráta (`learning_rate`), a `reward`, a `discount_factor`, és a jövőbeli Q-érték kombinációjával történik. Ez a formula garantálja, hogy az agent fokozatosan finomítja a Q-értékeket az optimális policy felé.

A tanulási paradigma dinamikusan kezeli az `exploration_rate`-t, amely az idő előrehaladtával exponenciálisan csökken (`exploration_decay`). Ez biztosítja, hogy az agent kezdetben felfedez, majd egyre inkább kihasználja a megszerzett tudást.

A `sensor_values` szótár definiálja az egyes szenzor adatokhoz társított numerikus értékeket, amelyek jellemzik az autó környezetét. Az állapot kinyerése a `get_state_from_sensors()` függvényben történik, ami a szenzorok által visszaadott adatok alapján számol egy összegzett állapotértéket. Ez az állapot kritikus fontosságú a Q-learning algoritmus számára, mivel az agent döntései és a tanulási folyamat eredményessége közvetlenül ettől függ. Ez a módszer lehetővé teszi, hogy az autó környezetének jellemzői, mint például a pálya, a pálya széle, fű, gyalogosok, vagy sár, diszkrét értékekké alakuljanak, amelyek összegezve egy egyedi állapotot képeznek. Ez az állapot használatos a tanulási algoritmusban, hogy döntéseket hozzon az adott helyzetben.

A `save_q_table()` és `load_q_table()` függvények segítségével lehetséges a tanulási folyamat során szerzett tudás mentése és későbbi betöltése. A Q-tábla mentése JSON formátumban történik, ami Pythonban könnyen kezelhető.

A `compute_reward()` függvény az autó aktuális állapotától függően számítja ki a jutalmat. A jutalmazási rendszer célja, hogy pozitív visszajelzést adjon kívánatos viselkedésért, és büntessen a nem kívánatos viselkedésért, ezzel segítve az agent tanulási folyamatát.

Az `apply_action_to_car()` függvényben a Q-learning agent által kiválasztott cselekvés kerül alkalmazásra az autóra. Ez a függvény a cselekvési lehetőségek között navigál, amelyek lehetnek előremenetel, fordulás jobbra vagy balra. Ez biztosítja, hogy az autó dinamikusan reagáljon a környezetéből származó változásokra az agent döntései alapján

6.3. Irányítások összehasonlítása

A gépi tanuláson alapuló irányítási rendszerek két megközelítése az alábbiakban van bemutatva: az egyik a megerősítéses tanulás, a másik a felügyelt tanulás. Mindkét rendszer szenzoradatokra támaszkodik a jármű irányításához, azonban a megközelítésük és alkalmazásuk jelentősen eltérő.

A Q-learning alapú rendszer megtanulja a környezetével való interakciót anélkül, hogy explicit célkitűzéseket kapna, pusztán a jutalmazási mechanizmuson keresztül. Ez a folyamat iteratív, ahol az agent folyamatosan frissíti a cselekvésekhez kapcsolódó várható jutalmak táblázatát (Q-tábla), amelyeket a különböző állapotokban tesztel. A tanulás során az agent fokozatosan csökkenti a véletlenszerű cselekvések arányát, miközben növeli a jutalmazás alapján optimálisnak vélt cselekvések alkalmazását. A Q-learning modell esetében az irányítási döntések a Q-tábla alapján történnek, amely prediktív elemzés helyett a korábbi tapasztalatokra támaszkodik.

A felügyelt tanuláson alapuló modell, mint az `apply_advanced_model_control()` funkció esetében, explicit tanító adatokon alapul, amelyek a helyes cselekvések mintáit tartalmazzák a bemeneti szenzoradatokhoz képest. Ez a módszer az adatokból közvetlenül megtanul egy modellt, amely a bemeneti szenzoradatokat a kívánt kimeneti irányítási parancsokhoz rendeli. A predikciók alapján az autó sebességét és irányát állítja be, ahol a modell kimenetei közvetlen utasításokká alakíthatóak át

- A Q-learning jobban alkalmazkodik a változó környezethez, mivel nem támaszkodik előre meghatározott címkézett adatokra. Ellenben, az SL modell teljesítménye közvetlenül függ a tanító adatok minőségétől és relevanciájától.
- Az SL modell hajlamos az overfittingre, ha a tanító adatok nem reprezentatívak. A Q-learning robusztusabb lehet, mivel folyamatosan frissíti a tanulmányait a környezeti visszajelzések alapján.
- A Q-learning implementációja általában bonyolultabb, mivel időbe telik, amíg a modell konvergál egy optimális politikához. Az SL modell viszont gyorsabban implementálható, ha rendelkezésre állnak megfelelő minőségű címkézett adatok.

- Az SL modell gyakran pontosabb irányítást nyújt rövid távon, mivel közvetlenül az adatokból tanul. A Q-learning lassabban tanul, de jobban képes kezelni az előre nem látható eseményeket hosszú távon.

7. Tanítási folyamat és kiértékelés

A gépi tanulási modellek fejlesztése során fontos szempont, hogy hogyan zajlik a tanítási folyamat, és hogyan értékeljük ki a tanult modellek teljesítményét. Ebben a részben a két, előző fejezetben tárgyalt SL és RL módszer tanítási és kiértékelési folyamatát hasonlítjuk össze.

7.1. SL modell tanítása

A felügyelt tanulás során a kód egy előre definiált, címkézett adathalmazon alapuló tanulást végez. A gépi tanulási modell itt egy neurális hálózat, amelyet arra képzünk ki, hogy a bemeneti szenzoradatok alapján előrejelezze a legmegfelelőbb vezetési döntéseket. A neurális hálózat több réteget és neuronokat tartalmaz. A tanulási folyamat során a hálózat a szenzoradatokat kapja bemenetként, és a jármű sebességének és irányának előrejelzését adja kimenetként. A hálózat súlyait a back-propagation algoritmus segítségével frissítjük, amely minimalizálja a kimenet és a tényleges címkék közötti eltérést. Ez a folyamat iteratív, minden egyes iterációval javul a modell teljesítménye.

A tanítás a `model.fit()` metódussal történik ahol megadjuk a tanító adatokat: `x_train`, `y_train`, valamint az iterációk számát : `epochs` és a kötegméretet: `batch_size`. További paramétereként a `validation_split` használata biztosítja, hogy az adathalmaz egy részét a tanulás során validációra használjuk, ami segít monitorozni a modell teljesítményét és a túltanulás esetleges jeleit. A `model.fit()` használata során `callbacks` paramétereket is használunk, például a `ReduceLROnPlateau` a tanulási rátát csökkenti, ha a modell nem javul tovább bizonyos epochokon keresztül, illetve az `EarlyStopping` korai leállást tesz lehetővé, ha a validációs veszteség nem javul további iterációk során.

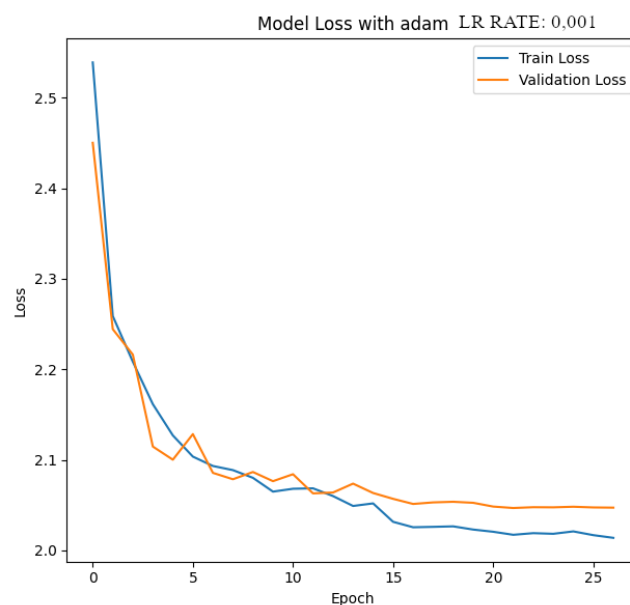
A tanítási folyamat befejeztével a `model.evaluate()` függvény segítségével kiértékeljük a modell teljesítményét a teszt adathalmazon. Ez ad objektív visszajelzést arról, hogy a modell mennyire jól teljesít új, látatlan adatokon.

7.1.1. SL modell eredményei

Az alábbiakban kielemezzük az Adam, RMSprop és SGD [3] optimalizálókkal készült tanulási görbéket. Minden grafikon [10] a tanítási veszteséget (Train Loss) és a validációs veszteséget (Validation Loss) mutatja az egyes epochok során. Az összes modell 100 epoch-ra volt állítva 0,001-es tanulásri rátával. Emellett megvizsgáljuk, hány epoch után állt meg a tanulás, és feltárjuk ennek lehetséges okait.

1. Adam optimizer: A 7.1-es ábra mutatja, hogy a tanulási folyamat alatt az Adam optimizer gyorsan csökkenti a veszteséget az első néhány epoch alatt, és utána lassul le. A tanulás körülbelül 25 epoch után áll meg a korai megállítás (early stopping) miatt. A tanítási veszteség folyamatosan csökken, de kisebb hullámzások láthatók. A validációs veszteség kezdetben meredeken csökken, majd a 10. epoch után stabilizálódik.

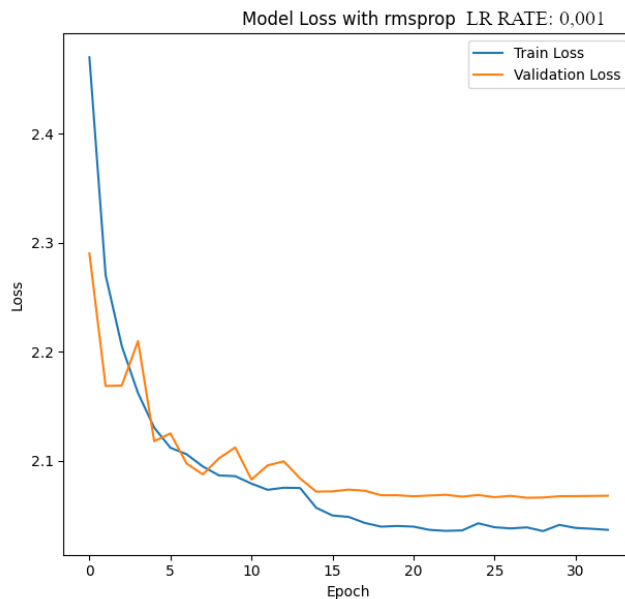
Következtetés: Az Adam optimalizáló hatékonyan tanul az adatokból, és gyors konvergenciát mutat. Az early stopping a stabilizálódó validációs veszteség miatt lépett életbe. A kisebb hullámzások a tanulási folyamat során általánosak az Adam optimalizálónál, mivel az adaptív tanulási rátákat használ.



7.1. ábra: Adam optimalizáló veszteség görbéje

2. RMSProp optimizer: A 7.2-es ábra mutatja, hogy a tanulási folyamat az RMSprop optimalizálással is gyorsan csökkenti a veszteséget az első néhány epoch alatt, de kicsit több ingadozással. A tanulás körülbelül 30 iteráció után áll meg a korai megállítás miatt. A tanítási veszteség folyamatosan csökken, de jelentősebb hullámzásokkal, mint az Adam esetében. A validációs veszteség kezdetben meredeken csökken, majd a 10. epoch után stabilizálódik, de ismét több ingadozással.

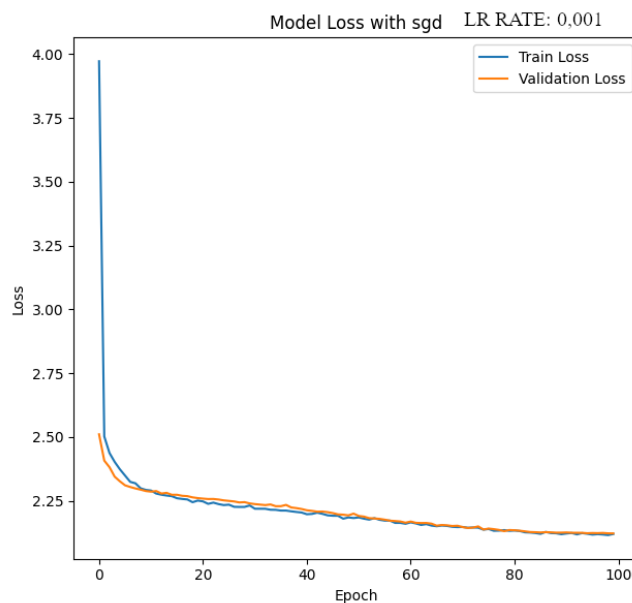
Következtetés: Az RMSprop optimalizáló szintén hatékony, de több ingadozással tanul. Az ingadozások az RMSprop belső működéséből fakadnak, amely az adaptív tanulási ráták mellett egy mozgóátlagot is használ. Az early stopping később lépett életbe, mint az Adam esetében, valószínűleg az ingadozások miatt.



7.2. ábra: RMSProp optimalizáló veszteség görbéje

3. SGD optimizer: A 7.3-as ábra mutatja, hogy a tanulási folyamat az SGD optimalizálóval lassabban csökkenti a veszteséget, de folyamatosan. A tanulás elérte a teljes 100 epochot, nincs korai megállítás. A tanítási veszteség folyamatosan csökken, nagyon sima görbével. A validációs veszteség hasonlóan folyamatosan csökken, a tanítási veszteséggel párhuzamosan.

Következtetés: Az SGD optimalizáló lassabb konvergenciát mutat, de nagyon stabil tanulási folyamatot biztosít. Mivel nincs early stopping, valószínű, hogy az SGD nem érte el a validációs veszteség minimális pontját, vagy a tanulási ráta túl kicsi volt ahhoz, hogy gyors konvergenciát érjen el. Az SGD stabilitása miatt kevésbé érzékeny az ingadozásokra, de lassabb tanulást eredményez.



7.3. ábra: SGD optimalizáló veszteség görbéje

A három optimalizáló összehasonlítása alapján, az adott esetre vonatkozóan a legjobb választásnak az Adam optimalizáló tűnik. Az Adam optimalizáló a leggyorsabb konvergenciát mutatja mind a tanítási, mind a validációs veszteség esetében. Már 25 epoch után stabilizálódik, ami időhatékony tanulást tesz lehetővé. A validációs veszteség viszonylag stabil, kisebb ingadozásokkal a tanulási folyamat során. Ez arra utal, hogy a modell jól általánosít az új adatokra. Az Adam optimalizálással használt early stopping mechanizmus jól működik, segítve a túltanulás elkerülését és a legjobb modell kiválasztását.

7.2. Q-learning tanítása és eredményei

1. Moving Average Reward: (7.5-ös ábra) Pozitív szakaszok: A jutalom mozgó átlaga többször is eléri a 100-as értéket, ami azt jelzi, hogy az agent gyakran helyesen cselekszik és a pályán marad. Az agent teljesítménye többször is jelentős visszaesést mutat, de gyorsan vissza tud térni a magasabb jutalmakhoz. Az ingadozások jelen vannak, de az agent képes stabilan magas jutalmakat elérni.

2. Total Reward: (7.4-es ábra) A grafikon 2000 epizódot mutat, ahol az agent teljesítménye változó, de gyakran eléri a maximális 100-as jutalmat. Az ingadozások jelentősek, az agent néha nagyon alacsony jutalmakat kap, ami az exploráció miatt is lehet. Az agent gyakran eléri a pozitív tartományt, ami azt mutatja, hogy a tanulási folyamat során egyre jobban teljesít.

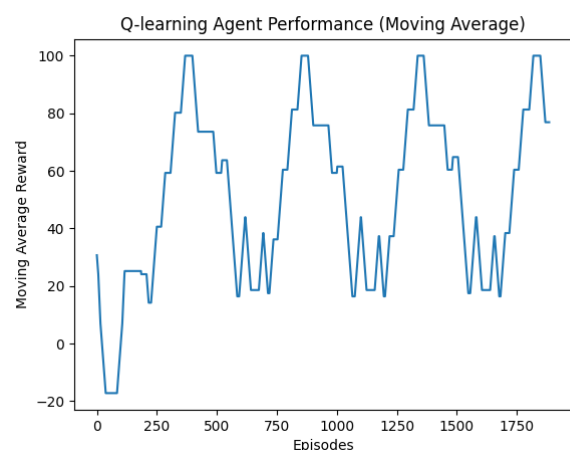
Átlagos Jutalom: 51.259. Az átlagos jutalom pozitív, ami jó jel arra, hogy az agent többnyire jól teljesít a tanulási folyamat során.

Tanulási paraméterek: `learning_rate=0.1`: viszonylag magas, ami segít az agentnek gyorsan alkalmazkodni, de nagyobb ingadozásokhoz vezethet. `discount_factor=0.95`: Jelzi, hogy az agent a jövőbeli jutalmakra is jelentős súlyt helyez. `exploration_rate=0.5` és `exploration_decay=0.99`: Az agent kezdetben gyakrabban explorál, de fokozatosan csökkenti az explorációt, hogy a tanulás során inkább a korábbi tapasztalatokra támaszkodjon. `min_exploration_rate=0.001`: Biztosítja, hogy az agent mindig fenntartsa egy kis mértékű explorációt, ami segíthet a hosszú távú optimalizációban.

Jutalmazási struktúra: Ütközés és fűre lépés = -100: Jelentős büntetés az ütközésért, ami segít elkerülni az akadályokat és a fűvet. Pálya Széle = -10: Kisebb büntetés a pálya elhagyásáért. Megállás = -50: Büntetés az autó megállásáért, hogy az agent folyamatos mozgásra törekedjen. Helyes Haladás = +100: Pozitív jutalom a helyes haladásért, ami ösztönzi az agentet a cél elérésére.



7.4. ábra: Az agent teljesítménye



7.5. ábra: Az agent mozgó átlag teljesítménye

7.3. Összegzés

Összehasonlítás: SL vs. RL

Gyorsaság és Stabilitás: SL (Adam optimalizáló): Gyors és stabil konvergenciát mutat, kevesebb ingadozással. RL: Bár az RL agent is képes elérni magas jutalmakat, jelentős ingadozások és visszaesések tapasztalhatóak, ami a tanulási folyamat kevésbé stabil jellegét mutatja.

Átlagos Teljesítmény: SL: Az átlagos teljesítmény stabil és magasabb pontosságot mutat, különösen az Adam optimalizálóval. RL: Az átlagos jutalom pozitív (51.259), de jelentős ingadozásokkal és visszaesésekkel.

Jutalmazási és Büntetési Mechanizmus SL: A felügyelet nélküli tanulás során a veszteség csökkentése az elsődleges cél, és a metrikák alapján történik a kiértékelés. RL: Az RL agent jutalmazási és büntetési mechanizmusokra támaszkodik, ami dinamikusabb és környezetfüggőbb tanulást eredményez.

Alkalmazási Terület SL: Jól használható strukturált adatokkal, ahol a cél egy meghatározott kimenet előrejelzése. RL: Alkalmas olyan dinamikus környezetekben, ahol a döntéshozatal és az akciók következményei időben változnak.

Összegzés:

Supervised Learning: Gyorsabb és stabilabb tanulási folyamat: Az Adam optimalizálóval a modell gyorsan és stabilan konvergál, kevesebb ingadozással.

Magasabb átlagos teljesítmény: Az SL modellek rendszerint magasabb pontosságot érnek el a tanulási folyamat végére. Jól definiált feladatokhoz: Ideális strukturált adatok és jól meghatározott kimenetek esetén.

Reinforcement Learning : Rugalmasabb tanulási mechanizmus: Az RL agentek képesek alkalmazkodni a dinamikus környezethez és a változó jutalmazási struktúrákhoz. Pozitív átlagos jutalom: Az agent pozitív átlagos jutalmat ér el, de jelentős ingadozásokkal. Dinamikus környezetekhez: Jobban alkalmazható olyan környezetekben, ahol a döntéshozatal és az akciók következményei időben változnak.

Összehasonlítás:

Teljesítmény: Az SL modellek általában magasabb és stabilabb teljesítményt nyújtanak rövidebb idő alatt, míg az RL agentek rugalmasságot és adaptivitást biztosítanak a dinamikus környezetekben.

Stabilitás: Az SL modellek stabilabb tanulási folyamatot mutatnak, míg az RL agentek jelentősebb ingadozásokat tapasztalhatnak a tanulás során.

Adaptivitás: Az RL agentek jobban alkalmazkodnak a környezet változásaihoz és a komplex feladatokhoz, ahol a visszacsatolási mechanizmusok fontos szerepet játszanak.

Mindkét megközelítésnek megvannak a maga erősségei és gyengeségei. A választás a konkrét alkalmazási terület és a probléma jellegétől függ. Az SL a strukturált, előrejelzési feladatokhoz, míg az RL a dinamikus, döntéshozatali problémákhoz nyújt jobb megoldást.

Irodalomjegyzék

- [1] Ian Goodfellow, Yoshua Bengio, Aaron Courville: Deep Learning, The MIT Press, 2016, ISBN: 978-0262035613
- [2] Hivatalos TensorFlow dokumentáció, <https://www.tensorflow.org/guide/basics>, utolsó megnyitás: 2024.05.14.
- [3] Hivatalos Keras dokumentáció, <https://keras.io/about/>, utolsó megnyitás: 2024.05.14.
- [4] Hivatalos Numpy dokumentáció, <https://numpy.org/doc/stable/>, utolsó megnyitás: 2024.05.14.
- [5] Hivatalos Pandas dokumentáció, https://pandas.pydata.org/docs/user_guide/index.html, utolsó megnyitás: 2024.05.14.
- [6] Hivatalos PyGame dokumentáció: <https://www.pygame.org/docs/>, utolsó megnyitás: 2024.05.14.
a. kép rotációáért felelős kód: <https://www.pygame.org/wiki/RotateCenter>
- [7] Huajin Tang, Kay Chen Tan, Zhang Yi: Neural Networks: Computational Models And Applications, Springer-Verlag Berlin Heidelberg, 2007, eBook ISBN: 978-3-540-69226-3
- [8] Richard S. Sutton, Andrew G. Barto: Reinforcement Learning: An Introduction, The MIT Press, 2014, 2015, ISBN: 978-0262039246
- [9] Q-learning in python, <https://www.geeksforgeeks.org/q-learning-in-python/>, utolsó megnyitás: 2024.05.14
- [10] Hivatalos Matplotlib dokumentáció, <https://matplotlib.org/>, utolsó megnyitás: 2024.05.14
- [11] Image by Freepik, https://www.freepik.com/free-vector/flat-illustration-people-looking-up_14602291.htm#query=people%20together&position=49&from_view=keyword&track=ais_user&uuid=bdd7c414-1cbe-4af6-9c8e-f6fe0e640d95
- [12] thypix splash 98, <https://thypix.com/wp-content/uploads/2021/03/splash-98.png>
- [13] supervised/unsupervised learning szemléltetés, <https://www.extrahop.com/blog/supervised-vs-unsupervised-machine-learning-for-network-threat-detection>
- [14] Reinforcement learning szemléltetés, <https://www.javatpoint.com/reinforcement-learning>
- [15] A simple illustration of how the backpropagation works by adjustments of weights, <https://www.geeksforgeeks.org/backpropagation-in-machine-learning/>
- [16] Underfitting and Overfitting | Source: SuperAnnotate, <https://medium.com/@praneshchoudhury/underfitting-and-overfitting-conceptsinmachinelearningee786a852e4c>
- [17] Neural Networks Architecture, <https://www.geeksforgeeks.org/artificial-neural-networks-and-its-applications/>

Nyilatkozat

Alulírott Simon Bence programtervező informatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2024. 05. 16.

Simon Bence

Köszönetnyilvánítás

Ezúton szeretnék köszönetet mondani témavezetőmnek, Dr. Dombi József Dánielnek, amiért idejét áldozta arra, hogy a dolgozat elkészülhessen. Szakmai tanácsai és belém vetett bizalma nélkül nem születhetett volna meg ez a dolgozat. Külön köszönettel tartozom amiért mindig nyitott volt az új ötleteimre, ha valamit máshogy képzeltem el.

Szeretném megköszönni a szaktársamnak Zsurzsa Dórinak, akivel együtt vészeltük át az egyetem fáradtságos óráit. Mindig próbáltuk a lehető legtöbbet kihozni magunkból és segíteni a másikat, amiben csak tudtunk és bíztattuk egymást nehéz helyzetekben.

Továbbá szeretnék köszönetet mondani Pintér Tamásnak, aki végig támogatott az egyetem alatt és tanácsaival képes voltam előre haladni. Hihetetlen matematika tudásával mindig segített, ha nem értettem az anyagot vagy elakadtam egy-egy bonyolultabb tárgynál. Tanulás közbeni/végi levezető játékok mindig fénypontot adtak az unalmasabb napoknak.

Akiknek a legnagyobb köszönettel tartozom, a családom. Testvérem Dávid és párja Szandra, valamint nényém Évi folyamatos biztatásukkal segítették tanulmányaimat. Szüleimnek, akik minden lépésemet támogatták és a legnehezebb döntéseknél is ott voltak velem és tanácsaikkal sikerült átvészelnem ezen időszakokat. Köszönöm a sok támogatást és szeretetet, amely minden nehézségen átsegített, a sok áldozatot, amit értem hoztatok, hogy eljussak idáig és elkészülhessen a szakdolgozatom.

Mellékletek

1. main.py
2. model.py
3. obstacles.py
4. model_control.py
5. automated_car.py
6. car.png
7. field.png
8. field_only.png
9. trackside.png
10. finish_line.png
11. grass.jpg
12. mud1.png
13. pedestrian3.png
14. automated_driving_data_full.csv
15. automated_trained_model_long_v1.18_rms_optimizer_5output_LR_0.001_jo.h5
16. automated_trained_model_long_v1.19_rms_optimizer_5output_LR_0.001.h5
17. q-table.json
18. rewards.json
19. 1sz_melleklet_sensor_collector.txt
20. 2sz_melleklet_vezerlopanel.txt

Fogalmak

TPU: Tensor Processing Unit

SGD: Stochastic Gradient Descent

RMSProp: Root Mean Square Propagation

ADAM: Adaptive Moment Estimation

NumPy: Numerical Python

ReLU: Rectified Linear Unit

MDP: Markov Decision Processes

MLP: Multilayer Perceptrons

SBP: Standard Back-Propagation

SL: Supervised Learning

RL: Reinforcement Learning

CC: Categorical Crossentropy

FNN: Feed-Forward Neural Network